

COMPLETE COURSE EDITION • MODULES 1 - 18

MySQL Complete Theory & Syntax Guide

From Database Fundamentals to ER Modelling



Table of Contents

MODULE 1 Database Fundamentals + DBMS Overview Data, Field, Record, Database · Limitations of File Systems · Advantages of DBMS · DBMS vs File System · Data Models Overview · Exploring RDBMS · Analysing Requirements	1
MODULE 2 Data Models + MySQL Installation & Setup Physical & Logical Models · Hierarchical · Network · Relational · Object · Object-Relational · Conceptual Models · MySQL vs Oracle vs DB2 · Installation · Authentication Method · Login & Password	4
MODULE 3 SQL Introduction + Databases History & Features of SQL · Client-Server Request Flow · SQL Command Types (DDL/DML/DQL/DCL/TCL) · ACID Properties · Creating/Altering/Deleting Databases · USE · SHOW DATABASES · Quick Reference Table	7
MODULE 4 Constraints Procedural vs Declarative Integrity · NOT NULL · UNIQUE · DEFAULT · CHECK · PRIMARY KEY · FOREIGN KEY / Referential Integrity	11
MODULE 5 Data Types & Working with Tables (DDL) CREATE DATABASE · USE · CREATE TABLE · Data Types · Constraints Recap · Composite Keys · DESCRIBE · ALTER (ADD/DROP/MODIFY/RENAME) · Generated Columns · DROP TABLE/DATABASE · TRUNCATE · CREATE TABLE ... LIKE	14
MODULE 6 DML — INSERT & UPDATE INSERT (Positional) · INSERT with Identity (AUTO_INCREMENT) · Named Columns & Multi-Row Insert · Creating a Table from Another Table · Copying Rows Between Tables · UPDATE · UPDATE with Computed Column	20
MODULE 7 DML — ALTER & DELETE ALTER — ADD/MODIFY Column · DELETE with WHERE · DELETE vs TRUNCATE vs DROP	23
MODULE 8 DQL — SELECT Fundamentals SELECT * vs Columns · Column Aliasing · WHERE Clause · ORDER BY · DISTINCT · GROUP BY with HAVING · Logical Order of Execution	25
MODULE 9 All Operators + DQL — Filtering & NULL Handling Arithmetic · Relational · Logical · Bitwise · Assignment Operators · BETWEEN · IN / NOT IN · LIKE / RLIKE · IS NULL · ISNULL() · EXISTS (preview) · Column Aliases	27
MODULE 10 Numeric + String Functions ROUND, CEIL, FLOOR, TRUNCATE(), POWER/POW, SQRT, ABS, MOD, RAND, SIGN, GREATEST, LEAST · UPPER/UCASE, LOWER/LCASE, LENGTH, CHAR_LENGTH, SUBSTRING, LEFT, RIGHT, CONCAT, CONCAT_WS, REPLACE, TRIM/LTRIM/RTRIM, LPAD, RPAD, LOCATE/INSTR, SUBSTRING_INDEX	31
MODULE 11 Date/Time + Aggregate + System Functions NOW, CURDATE, CURTIME, YEAR/MONTH/DAY, HOUR/MINUTE/SECOND, EXTRACT, DAYNAME, DATEDIFF, TIMEDIFF, DATE_ADD, DATE_SUB, DATE_FORMAT, STR_TO_DATE · Aggregates: COUNT/SUM/AVG/MAX/MIN · System: VERSION(), DATABASE(), USER(), CONNECTION_ID(), LAST_INSERT_ID()	35
MODULE 12 Normalization (1NF, 2NF, 3NF)	39

Why Normalize · Insertion/Update/Deletion Anomalies · First Normal Form · Second Normal Form · Third Normal Form — Practical Normalization

MODULE 13

Set Operators + Joins Part 1

UNION · UNION ALL · INTERSECT · EXCEPT · INNER JOIN · EQUI JOIN · NATURAL JOIN · NON-EQUI JOIN

42

MODULE 14

Joins Part 2 (Outer & Cross Joins)

LEFT OUTER JOIN · RIGHT OUTER JOIN · FULL OUTER JOIN (simulation) · CROSS JOIN · Anti-Join Patterns · SELF JOIN · Multi-Table Joins

45

MODULE 15

Subqueries

Single-Row · Multi-Row (IN) · ANY / SOME / ALL · Derived Tables · Correlated Subqueries · Nested Subqueries · EXISTS / NOT EXISTS

48

MODULE 16

Views + Transaction Management

CREATE VIEW · Advantages/Disadvantages of Views · CREATE OR REPLACE VIEW · Querying/Describing Views · Updatable Views · DROP VIEW · AUTOCOMMIT · START TRANSACTION/BEGIN · COMMIT · ROLLBACK · SAVEPOINT

51

MODULE 17

Stored Procedures and Triggers

Advantages of Stored Procedures · DELIMITER · CREATE PROCEDURE · IN/OUT/INOUT · CALL · Variables · IF...ELSE · DROP PROCEDURE · CREATE TRIGGER · BEFORE/AFTER · OLD/NEW · FOR EACH ROW · DROP TRIGGER

55

MODULE 18

ER Modelling (Entities, Relationships, Diagrams)

Entities & Attributes · Relationships & Cardinality · Keys in ER · Attribute Types (Composite/Multi-valued/Derived) · Weak Entities · Notation · Full ER Diagrams

60

MODULE 1

Database Fundamentals + DBMS Overview

The conceptual foundation everything else in this guide sits on top of — what data actually is, why we organize it the way we do, and why we use a DBMS instead of plain files.

1.1 What is Data

Definition: Data is any raw fact or value that, on its own, has no meaning until it is processed or given context — e.g. the number 25, the word "Ravi", or the date 2004-08-10.

RULES & NOTES

- Data becomes **information** once it is organized/processed to have meaning — e.g. "Ravi is 25 years old" is information built from two raw data values.
- In a database, data is stored as rows and columns inside tables.

EXAMPLE

```
25          -- just a number (data)
"Ravi"      -- just a name (data)
"Ravi is 25 years old" -- information (data + context)
```

1.2 Field

Definition: A field (also called a column or attribute) is a named category of data that every record in a table shares — e.g. every student record has a NAME field, a CLASS field, and so on.

RULES & NOTES

- Every field has a name and a data type (e.g. NAME VARCHAR(30), AGE TINYINT).
- All values in one field/column across a table share the same data type.

EXAMPLE

```
-- In the STUDENTS table, each of these is a field:
STDID, NAME, GENDER, CLASS, AGE, DATE_OF_BIRTH
```

1.3 Record

Definition: A record (also called a row or tuple) is one complete set of related field values — a single entry in a table.

RULES & NOTES

- A table is essentially a collection of records that all share the same fields.
- Tables should normally have a primary key, so each record can be uniquely identified.

EXAMPLE

```
INSERT INTO students VALUES (101,"ravi","male",6,11,"2015-01-01");
-- This entire row is ONE record: STDID=101, NAME='ravi', GENDER='male', CLASS=6, AGE=11, DOB='2015-01-01'
```

1.4 What Is a Database?

Definition: A database is an organized, structured collection of related data, stored electronically so it can be efficiently created, read, updated, and deleted (CRUD).

RULES & NOTES

- A database is managed by a **DBMS** (Database Management System) — the software layer that lets users/applications interact with the data.
- A single database (e.g. classicmodels) can contain many related tables (customers, orders, employees, etc.).

EXAMPLE

```
CREATE DATABASE SCHOOL;
USE SCHOOL;
-- SCHOOL is now a database that can hold the STUDENTS and MARKS tables
```

1.5 Limitations of File Management System

Definition: Before DBMS existed, data was stored directly in flat files (like text or spreadsheet files) managed by individual applications. This approach — the File Management System (FMS) — has serious structural weaknesses at scale.

RULES & NOTES

- **Data redundancy** — the same data (e.g. a customer's address) often gets duplicated across multiple files.
- **Data inconsistency** — because of redundancy, updating one copy but not another leaves contradictory data.
- **Difficulty in accessing data** — every new query/report often needs new application code; there's no query language.
- **Limited data sharing & concurrency issues** — files are hard to safely share across multiple users/programs at once.
- **No proper security** — file-level permissions are coarse; you can't easily restrict access to specific fields.
- **Poor data integrity** — no built-in way to enforce rules like "age must be positive" or "foreign key must exist".
- **No backup/recovery guarantees** — crash recovery must be built manually into each application.

1.6 Advantages of DBMS

Definition: A DBMS solves the problems of plain file systems by centralizing data management with built-in structure, rules, and tools.

RULES & NOTES

- **Reduced redundancy** — normalization (Module 12) minimizes duplicate data.
- **Data consistency & integrity** — constraints (PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL) enforce valid data automatically.
- **Easy data access** — SQL (a declarative query language) lets you ask for exactly the data you want without writing custom retrieval code.
- **Data sharing & concurrency control** — multiple users can safely read/write at once (transactions, locking).
- **Security** — user accounts, privileges (GRANT/REVOKE via DCL), and views can restrict what each user sees.
- **Backup & recovery** — DBMS software provides built-in mechanisms to recover from crashes (transaction logs).
- **Data independence** — the way data is physically stored can change without breaking the applications that use it.

1.7 DBMS vs File System

Definition: A side-by-side comparison of the two approaches to storing and managing data.

Aspect	File System	DBMS
Redundancy	High — same data repeated in many files	Minimized via normalization
Data access	Requires custom program code	Declarative queries (SQL)
Integrity rules	Enforced manually in each application	Enforced centrally via constraints
Concurrency	Difficult, error-prone	Built-in locking/transactions
Security	OS-level file permissions only	Fine-grained user/role privileges
Backup/Recovery	Manual	Built-in (logs, COMMIT/ROLLBACK)

1.8 Data Models Overview

Definition: A data model is a conceptual way of organizing and describing how data elements relate to one another. It's the "blueprint" a DBMS is built around. Covered in detail in Module 2.

RULES & NOTES

- Common types: Hierarchical, Network, Relational, Object, Object-Relational.
- MySQL is a **Relational** DBMS (RDBMS) — data is organized into related tables.

1.9 Exploring Relational DBMS

Definition: A Relational DBMS (RDBMS) stores data in tables (relations) made of rows and columns, where relationships between tables are expressed through shared key values (primary key / foreign key) rather than physical links.

RULES & NOTES

- Based on E.F. Codd's relational model (1970) — data as mathematical "relations" (sets of tuples).
- Popular RDBMS: MySQL, PostgreSQL, Oracle, SQL Server, IBM DB2.

- Tables relate to each other via keys — e.g. `ORDERS.CUSTOMERNUMBER` references `CUSTOMERS.CUSTOMERNUMBER`.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER,CUSTOMERNAME,ORDERNUMBER FROM CUSTOMERS C
INNER JOIN ORDERS O ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER;
-- CUSTOMERS and ORDERS are two separate relations, joined via a shared key
```

1.10 Analysing Requirements

Definition: Before designing any database, you first analyze what the system needs to store and do — identifying the real-world entities, the data each one needs, and how they relate — which then feeds into ER Modelling (Module 18) and Normalization (Module 12).

RULES & NOTES

- Typical steps: gather requirements → identify entities (Student, Order, Customer) → identify attributes (name, class, order date) → identify relationships (a Student "has" Marks; a Customer "places" Orders) → draw an ER diagram → normalize → implement as tables.
- One example from practice: the SCHOOL database's requirement was "track students and their marks per subject", which led naturally to a STUDENTS table and a related MARKS table linked by STDID.

See Module 18 for the full worked design.

Data Models + MySQL Installation & Setup

How different database systems model data, where MySQL fits among them, and how to get MySQL running on your machine.

2.1 Physical & Logical Data Models

Definition: A **logical data model** describes data structures and relationships independent of any specific database software (what entities exist, what attributes they have). A **physical data model** describes how that logical design is actually implemented in a specific DBMS (actual tables, data types, indexes, storage engine).

RULES & NOTES

- Logical model: "A Student has a Name and a Class." Physical model: "NAME VARCHAR(30), CLASS TINYINT UNSIGNED in a MySQL InnoDB table."
- This separation is a core reason DBMS provide *data independence* (Module 1.6).

2.2 Hierarchical (HDBMS)

Definition: An early data model that organizes data as a tree — each record has exactly one parent, and parent-child relationships are strictly one- to-many (e.g. IBM's IMS).

RULES & NOTES

- Fast for pure parent→child lookups but rigid — a record cannot easily have two parents.
- Example real-world shape: a Company → Department → Employee tree.

2.3 Network (NDBMS)

Definition: An evolution of the hierarchical model that allows a record to have multiple parents, represented as a graph of records connected by explicit pointers/links (e.g. IDMS).

RULES & NOTES

- More flexible than hierarchical (supports many-to-many), but is primarily navigational — applications typically follow explicit pointers/links rather than issuing declarative queries the way SQL does.

2.4 Relational (RDBMS)

Definition: Organizes data into independent tables (relations) linked by key values instead of physical pointers — the model MySQL, Oracle, PostgreSQL, and SQL Server all use.

RULES & NOTES

- Foreign keys represent and enforce a referential relationship between tables — but JOIN itself is a query operation that can combine rows on ANY matching condition, whether or not a FOREIGN KEY constraint actually exists (Modules 13-14).
- Supports a declarative query language (SQL) so you describe *what* you want, not *how* to navigate to it.

EXAMPLE

```
-- classicmodels: CUSTOMERS and ORDERS are independent tables,
-- related only via the shared CUSTOMERNUMBER key — the relational model in action.
```

2.5 Object (ODBMS)

Definition: Stores data as objects, similar to object-oriented programming — with classes, inheritance, and encapsulated behavior — instead of plain rows and columns.

RULES & NOTES

- Useful when the application's object model maps awkwardly onto tables (e.g. complex nested/CAD data).
- Much less common in mainstream business systems than RDBMS.

2.6 Object-Relational (ORDBMS)

Definition: A hybrid model — a relational database core (tables, SQL) extended with object-oriented features like custom data types, inheritance between tables, and stored objects/arrays in columns.

RULES & NOTES

- PostgreSQL is a well-known ORDBMS. MySQL is primarily a "pure" RDBMS with some JSON/object-like extensions (e.g. the JSON data type) rather than a full ORDBMS.

2.7 Conceptual Models

Definition: A high-level, technology-independent model that captures WHAT data an organization needs and how the real-world entities relate — before deciding on any specific data model or DBMS. The Entity-Relationship (ER) model (Module 18) is the most common conceptual model.

RULES & NOTES

- Design flow: Conceptual model (ER diagram) → Logical model (normalized tables/columns) → Physical model (actual CREATE TABLE in MySQL).

2.8 MySQL vs Oracle vs DB2 — Advantages & Drawbacks

Database	Advantages	Drawbacks
MySQL	Free/open-source, huge community, fast for read-heavy web apps, easy to learn	Historically weaker advanced analytics/window-function support in older versions; fewer enterprise features than Oracle
Oracle	Extremely robust, advanced enterprise features (PL/SQL, partitioning, high scalability)	Expensive licensing, steeper learning curve
IBM DB2	Strong mainframe/enterprise integration, mature performance tuning tools	Expensive, smaller community, less common outside large enterprises

2.9 Installation

Definition: The process of downloading and setting up the MySQL Server (and typically MySQL Workbench as a GUI client) on your machine so you can create databases and run SQL.

RULES & NOTES

- Download MySQL Community Server from the official MySQL site (dev.mysql.com/downloads).
- The installer lets you choose a setup type (Developer Default is common for learning) and will also offer MySQL Workbench, MySQL Shell, and sample databases.
- During installation you configure the port (default 3306), the authentication method, and the root password.

2.10 Authentication Method (Auth Plugin)

Definition: During installation, MySQL asks you to choose an authentication method for the root account.

RULES & NOTES

- **Strong Password Encryption (caching_sha2_password)** — the modern default since MySQL 8.0; requires MySQL 8.0+ compatible connectors.
- **Legacy Authentication (mysql_native_password)** — was widely used through MySQL 8.0, but is deprecated as of 8.0.34, **disabled by default in MySQL 8.4**, and fully removed in MySQL 9.0. It can still be explicitly re-enabled in 8.4 for legacy compatibility if needed.
- This is distinct from "Windows Authentication" in SQL Server. Standard MySQL Community Server setups always authenticate through MySQL's own internal accounts — there's no OS-integrated login by default. (MySQL Enterprise Edition on Windows does offer an optional Windows- authentication plugin, but that's an Enterprise-only feature, not part of the standard install this guide covers.)

2.11 Login & Password

Definition: Once installed, you connect to the MySQL server using a username/password pair — most commonly the root superuser while learning.

SYNTAX

```
mysql -u root -p
-- then enter the password when prompted
```

RULES & NOTES

- -u specifies the username; -p prompts for the password interactively (safer than typing it inline).
- In MySQL Workbench, you instead create a "Connection" with host, port, username, and password saved together.
- After logging in, running a script typically starts with selecting a database: USE classicmodels;

EXAMPLE

```
mysql -u root -p  
Enter password: *****  
mysql> USE classicmodels;
```

SQL Introduction + Databases

What SQL is, the five command categories that structure this entire guide, and the basic database-level commands you already use at the top of every script.

3.1 History & Features of SQL

Definition: SQL (Structured Query Language) is a declarative language for defining, manipulating, and querying data in a relational database. It originated from IBM's research in the early 1970s (based on Codd's relational model) and became an ANSI/ISO standard in 1986.

RULES & NOTES

- **Declarative** — you describe *what* data you want, not the step-by-step procedure to get it (unlike general-purpose programming languages).
- Widely standardized, but every vendor (MySQL, Oracle, SQL Server, PostgreSQL) has its own extensions/dialect.
- Key features: set-based operations, built-in aggregate functions, constraints for data integrity, and support for transactions.

Year	Milestone
1970	E. F. Codd published the relational model.
1974	SEQUEL developed at IBM by researchers — later renamed SQL.
1979	Oracle released the first commercial SQL database.
1986	ANSI (American National Standards Institute) standardized SQL.
1990s onward	SQL standards and implementations continued to evolve.

3.1b Client-Server Request Flow

Definition: The basic path a request takes when you interact with a database-backed application — the User never talks to the Database directly; the Application sits in between.

Step	Flow
1	User sends a request to the Application (App).
2	Application forwards the request to the Database (DB).
3	Database sends a response back to the Application.
4	Application returns the response to the User.

User → Application → Database → (response) → Application → User

3.2 MySQL Types of SQL Commands: DDL

Definition: SQL commands are grouped into five categories based on what they do. **DDL (Data Definition Language)** defines/modifies the structure of database objects — covered fully in Modules 4-5 & 7.

RULES & NOTES

- DDL commands: CREATE, ALTER, DROP, TRUNCATE.
- Most DDL statements cause an **implicit commit** in MySQL — they end any active transaction before and after running, so their effects cannot be undone with a normal ROLLBACK.

EXAMPLE

```
CREATE TABLE STUDENTS(STDID INT PRIMARY KEY AUTO_INCREMENT, NAME VARCHAR(30));
```

3.3 DML

Definition: DML (Data Manipulation Language) manipulates the actual data (rows) inside tables — covered fully in Modules 6-7.

RULES & NOTES

- DML commands: INSERT, UPDATE, DELETE.
- Unlike DDL, DML changes CAN be rolled back inside a transaction (Module 16).

EXAMPLE

```
UPDATE marks SET s1 = s1-5 WHERE stdid=101;
```

3.4 DQL

Definition: DQL (Data Query Language) retrieves/reads data without changing it — covered fully in Modules 8-9 and used throughout Modules 10-15.

RULES & NOTES

- SQL only has one DQL command — SELECT — but it supports many clauses (WHERE, GROUP BY, ORDER BY, etc).

EXAMPLE

```
SELECT * FROM CUSTOMERS WHERE STATE IS NULL;
```

3.5 DCL

Definition: DCL (Data Control Language) manages permissions and access rights — who is allowed to do what in the database.

SYNTAX

```
GRANT privilege ON object TO user;
REVOKE privilege ON object FROM user;
```

RULES & NOTES

- DCL commands: GRANT, REVOKE.
- Not used directly in the examples in this guide (which all run as a single root/admin user), but essential in real multi-user systems.

EXAMPLE

```
-- Assumes the 'reporting_user' account already exists (created via CREATE USER):
GRANT SELECT, INSERT ON school.students TO 'reporting_user'@'localhost';
REVOKE INSERT ON school.students FROM 'reporting_user'@'localhost';
```

3.6 TCL

Definition: TCL (Transaction Control Language) manages transactions — groups of DML statements that succeed or fail together — covered fully in Module 16.

RULES & NOTES

- TCL commands: COMMIT, ROLLBACK, SAVEPOINT, START TRANSACTION/BEGIN.

EXAMPLE

```
begin;
update accounts set amount = amount-2000 where acc_no=101;
commit;
```

3.6b ACID Properties

Definition: ACID is the set of four guarantees a properly-managed transaction provides — the whole reason TCL (Module 3.6) and transactions (Module 16) exist in the first place.

Property	Meaning
Atomicity	For a transactional storage engine such as InnoDB, a transaction's changes are committed as one unit or rolled back as one unit — either every statement in it takes effect, or none do (subject to statements that cause an implicit commit, Module 3.2).
Consistency	A transaction moves the database from one valid state to another, respecting all defined integrity rules (constraints) as well as the application's own business rules.
Isolation	Concurrent transactions are isolated from each other according to the configured isolation level — one transaction shouldn't

	see another's uncommitted changes.
Durability	Once a transaction is committed, its changes persist even after a system crash or power failure.

This SAVEPOINT/ROLLBACK TO pattern is a direct illustration of Atomicity: it exists precisely so a multi-step money transfer either fully applies or fully reverts — never leaving one account debited without the other being credited.

3.7 Creating (Databases)

Definition: Creates a new, empty database as a container for tables and other objects.

SYNTAX

```
CREATE DATABASE [IF NOT EXISTS] database_name;
```

EXAMPLE

```
CREATE DATABASE SCHOOL;
CREATE DATABASE IT_COMPANY;
```

3.8 Altering (Databases)

Definition: Changes database-level properties, most commonly its default character set or collation (rare in day-to-day practice compared to ALTER TABLE).

SYNTAX

```
ALTER DATABASE database_name CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

RULES & NOTES

- Does not rename a database — MySQL has no direct RENAME DATABASE; you must create a new one and move the tables.

3.9 Deleting Databases

Definition: Permanently removes an entire database along with every object inside it — tables, views, procedures, triggers, functions, and events.

SYNTAX

```
DROP DATABASE [IF EXISTS] database_name;
```

RULES & NOTES

- Irreversible — there is no undo once committed.

EXAMPLE

```
DROP DATABASE IF EXISTS test_db;
```

3.10 USE

Definition: Selects a database as the current/active database for all subsequent statements in the session.

SYNTAX

```
USE database_name;
```

EXAMPLE

```
use classicmodels;
use school;
use bank;
```

3.11 SHOW DATABASES

Definition: Lists all databases currently present on the MySQL server you're connected to.

SYNTAX

```
SHOW DATABASES;
```

RULES & NOTES

- Useful as a first check before CREATE DATABASE or USE, to confirm a database exists (or doesn't yet).
- Related: SHOW TABLES; lists tables inside the currently active database.

EXAMPLE

```
SHOW DATABASES;
-- Example output – your server may show additional built-in schemas (like
-- 'information_schema', 'performance_schema', 'mysql') alongside your own:
-- +-----+
-- | Database          |
-- +-----+
-- | classicmodels     |
-- | school            |
-- | bank              |
-- +-----+
```

3.12 Quick Reference — SQL Command Categories & Auto-commit

A consolidated summary of every command category covered across this guide, and whether it auto-commits — pulling together DDL (Module 5), DML (Modules 6-7), DQL (Modules 8-9), DCL (Module 3.5), and TCL (Modules 3.6 & 16) into one table.

Category	Commands	Auto-commit?
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME	Yes (auto-committed)
DML	INSERT, UPDATE, DELETE	No (not auto-committed)
DQL	SELECT	N/A (read-only)
DCL	GRANT, REVOKE	Yes
TCL	START TRANSACTION/BEGIN, COMMIT, ROLLBACK, SAVEPOINT	N/A (controls transactions)

Constraints

Rules enforced on table columns to guarantee data accuracy and integrity — the safety net that stops bad data from ever entering a table.

4.1 Procedural Integrity Constraints

Definition: Integrity rules enforced through application code or procedural logic (e.g. inside a stored procedure or trigger) rather than declared directly on the table definition.

RULES & NOTES

- Example: a trigger that checks a condition before allowing an INSERT/UPDATE (see Module 17, Triggers).
- More flexible than declarative constraints (can express complex, multi-table business rules) but more work to write and maintain, and easier to accidentally bypass.

EXAMPLE

```
-- An AFTER UPDATE trigger enforces a procedural rule:
-- "every balance change on accounts must be logged into transactions"
create trigger after_transaction_trigger after update on accounts for each row
begin
insert into transactions(account,type,amount)
values(old.acc_no, if(old.amount>new.amount,"debit","credit"), abs(old.amount-new.amount));
end;
```

4.2 Declarative: NOT NULL

Definition: A **declarative** constraint — defined directly in the column definition — that requires a column to always have a value; it can never store NULL.

SYNTAX

```
column_name datatype NOT NULL
```

EXAMPLE

```
NAME varchar(30) NOT NULL,
GENDER varchar(30) NOT NULL
```

4.3 UNIQUE

Definition: Ensures every value in a column (or combination of columns) is different across all rows — no duplicates allowed, but unlike PRIMARY KEY, NULL values ARE allowed (and multiple NULLs don't count as duplicates of each other).

SYNTAX

```
column_name datatype UNIQUE
-- or as a table-level constraint:
UNIQUE (col1, col2)
```

RULES & NOTES

- A table can have many UNIQUE constraints, but only one PRIMARY KEY.
- Commonly used for things like email addresses or usernames.

EXAMPLE

```
CREATE TABLE users(
id INT PRIMARY KEY AUTO_INCREMENT,
email VARCHAR(100) UNIQUE,
username VARCHAR(50) UNIQUE
);
```

4.4 DEFAULT

Definition: Sets an automatic fallback value for a column, used whenever an INSERT statement does not explicitly supply a value for it.

SYNTAX

```
column_name datatype DEFAULT value
```

RULES & NOTES

- DEFAULT NULL makes explicit that NULL is the fallback (and implicitly that the column is nullable).
- DEFAULT CURRENT_TIMESTAMP is a very common pattern for "created at" columns.

EXAMPLE

```
created_at timestamp default current_timestamp  
addressLine2 VARCHAR(50) DEFAULT NULL
```

4.5 CHECK

Definition: Validates that inserted/updated rows satisfy a condition. MySQL accepts the row when the CHECK expression evaluates to TRUE or UNKNOWN — it only rejects the row when the expression evaluates to FALSE.

SYNTAX

```
column_name datatype CHECK(condition)
```

RULES & NOTES

- Fully enforced starting MySQL 8.0.16+ (earlier versions parsed but ignored CHECK).
- **Important gotcha:** a NULL value makes most conditions evaluate to UNKNOWN, not FALSE — so CHECK(AGE>5) does NOT reject a row where AGE is NULL. If NULL must be disallowed, add NOT NULL separately — CHECK alone won't do it.

EXAMPLE

```
ALTER TABLE STUDENTS ADD COLUMN AGE TINYINT CHECK(AGE>5);  
-- INSERT ... (AGE=3) -- REJECTED: 3>5 is FALSE  
-- INSERT ... (AGE=NULL) -- ACCEPTED: NULL>5 is UNKNOWN, not FALSE
```

4.6 Primary Key

Definition: Uniquely identifies every row in a table. Combines the guarantees of NOT NULL and UNIQUE, and a table can have only one.

SYNTAX

```
column_name datatype PRIMARY KEY  
-- composite version:  
PRIMARY KEY (col1, col2)
```

RULES & NOTES

- Often paired with AUTO_INCREMENT to generate IDs automatically.
- Can be a single column or a composite of multiple columns (Module 5.5b).

EXAMPLE

```
STDID INT PRIMARY KEY AUTO_INCREMENT
```

4.7 Foreign Key / Referential Integrity

Definition: Links one or more child-table columns to corresponding columns in a parent table, guaranteeing that any value stored must already exist there — this is what "referential integrity" means. In common design, the referenced parent column is a PRIMARY KEY or UNIQUE key (it isn't required to be specifically the PRIMARY KEY).

SYNTAX

```
FOREIGN KEY (column) REFERENCES other_table(column)
```

RULES & NOTES

- Prevents "orphan" rows — e.g. a MARKS row can't reference a STDID that doesn't exist in STUDENTS.

- By default, MySQL blocks deleting/updating a referenced row if child rows still exist, unless `ON DELETE CASCADE/ON UPDATE CASCADE` is specified.

EXAMPLE

```
CREATE TABLE MARKS(  
  STDID INT,  
  S1 INT, S2 INT, S3 INT,  
  PERCENTAGE DECIMAL(4,2),  
  FOREIGN KEY (STDID) REFERENCES STUDENTS(STDID));
```

MODULE 5

Data Types & Working with Tables (DDL)

Building the physical structure of a database: databases, tables, data types, and every flavor of ALTER TABLE.

5.1 CREATE DATABASE

Definition: Creates a new database (a logical container that holds tables, views, procedures, etc.).

SYNTAX

```
CREATE DATABASE database_name;
```

RULES & NOTES

- Database names must be unique on the server.
- Use IF NOT EXISTS to avoid an error if it already exists.

EXAMPLE

```
CREATE DATABASE SCHOOL;
```

5.2 USE

Definition: Selects a database as the current/active database for all subsequent statements in the session.

SYNTAX

```
USE database_name;
```

EXAMPLE

```
USE SCHOOL;
```

5.3 CREATE TABLE

Definition: Creates a new table by defining its columns, data types, and constraints.

SYNTAX

```
CREATE TABLE [IF NOT EXISTS] table_name (  
    column1 datatype [constraint],  
    column2 datatype [constraint],  
    ...  
);
```

RULES & NOTES

- IF NOT EXISTS prevents an error if the table already exists (but does NOT update structure if it already exists differently).
- Column order in the definition determines the table's declared/displayed column order.

EXAMPLE

```
CREATE TABLE IF NOT EXISTS telugu_movies(  
    id INT, movie_name VARCHAR(30), director VARCHAR(30), duration INT  
);
```

5.4 Common Data Types

Definition: Data types define what kind of value a column can store and how much storage it uses.

SYNTAX

```
INT | SMALLINT | TINYINT | BIGINT | DECIMAL(p,s) | VARCHAR(n) | TEXT | MEDIUMTEXT |  
MEDIUMBLOB | DATE | TIMESTAMP | ENUM(...)
```

RULES & NOTES

- **INT** — whole numbers. **TINYINT** — very small integers (age, flags). **SMALLINT** — a step up from TINYINT.
- **BIGINT** — very large whole numbers; good for large IDs, counters, or quantities. For monetary values, **DECIMAL** is normally preferred (**BIGINT** stores no fractional part).
- **DECIMAL(p,s)** — exact fixed-point number; p = total digits, s = digits after decimal.
- **VARCHAR(n)** — variable-length text up to n characters.
- **TEXT / MEDIUMTEXT** — long text types with large but finite maximum sizes (not literally unbounded — see the full size table in Module 5.4b).
- **MEDIUMBLOB** — large binary data (images, files).
- **DATE** — YYYY-MM-DD. **TIMESTAMP** — date+time, often auto-filled with `DEFAULT CURRENT_TIMESTAMP`.
- **ENUM('a','b',...)** — restricts a column to a fixed predefined list of string values.
- **UNSIGNED** — used after an integer type to disallow negative values.

EXAMPLE

```
CLASS TINYINT UNSIGNED NOT NULL,
PERCENTAGE DECIMAL(5,2),
GENDER ENUM('MALE','FEMALE'),
AMOUNT BIGINT
```

5.4b Full Datatype Reference Table (Numeric, Date/Time, String, Special)

MySQL datatypes fall into several groups. Module 5.4 covered the ones you'll use constantly — this reference table covers most of the rest you'll run into, though MySQL also has a few even more specialized types (e.g. **BIT** and spatial/geometry types) not shown here since they're rarely needed at this level.

NUMERIC DATATYPES

Datatype	Category	Storage
TINYINT	Integer	1 byte
SMALLINT	Integer	2 bytes
MEDIUMINT	Integer	3 bytes
INT	Integer	4 bytes
BIGINT	Integer	8 bytes
FLOAT	Approximate numeric	4 bytes (ordinary FLOAT)
DOUBLE	Double-precision approximate	8 bytes
DECIMAL	Exact fixed-point	Varies with precision & scale

FLOAT/DOUBLE vs DECIMAL: **FLOAT** and **DOUBLE** are approximate — fast, but can introduce tiny rounding errors. **DECIMAL** is exact — always use it for money (like a `PERCENTAGE DECIMAL(5,2)` column), never **FLOAT/DOUBLE**.

DATE/TIME DATATYPES

Datatype	Stores	Range
DATE	Date value	1000-01-01 to 9999-12-31
TIME	Time value	-838:59:59 to 838:59:59
DATETIME	Date and time	1000-01-01 00:00:00 to 9999-12-31 23:59:59
TIMESTAMP	Date and time	1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
YEAR	Year value	1901 to 2155 (also 0000)

DATETIME vs TIMESTAMP: both store date+time, but **TIMESTAMP** has a much narrower range (1970-2038) and is stored relative to UTC, converting to the session's timezone on read — useful when you want that automatic timezone conversion. **DATETIME** has no timezone conversion and a far wider range, so it's the safer choice whenever a date might need to fall outside 1970-2038 (e.g. birth dates, future-dated records). A `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` pattern works well for **TIMESTAMP** specifically because "now" is always safely inside that range — not because the data "doesn't need to survive" past 2038.

STRING DATATYPES

Datatype	Description
CHAR(n)	Fixed-length string. n = 0-255 characters — always stores exactly n characters, padding with spaces.
VARCHAR(n)	Variable-length string. Storage = actual length + 1 or 2 length-prefix bytes.
BINARY(n)	Fixed-length binary string. n = 0-255 bytes.
VARBINARY(n)	Variable-length binary string.
TINYTEXT	Text up to 255 bytes.
TEXT	Text up to 65,535 bytes (64 KB).
MEDIUMTEXT	Text up to 16,777,215 bytes (16 MB).
LONGTEXT	Text up to 4,294,967,295 bytes (4 GB).
TINYBLOB / BLOB / MEDIUMBLOB / LONGBLOB	Binary large objects — same size tiers as the TEXT family (BLOB = Binary Large Object).

CHAR vs VARCHAR: CHAR is fixed-width (fast, but wastes space padding short values) — good for things that are always the same length, like a 2-letter country code. Note that MySQL strips trailing spaces from CHAR values when you retrieve them (unless the unusual PAD_CHAR_TO_FULL_LENGTH mode is enabled), so a SELECT won't show you the padding even though it's stored internally. VARCHAR only stores what you actually use — better for names, addresses, anything variable-length. NAME VARCHAR(30) columns are the right choice since names vary in length.

ENUM AND SET

```
ENUM('male','female')      -- stores exactly ONE value from a predefined list
SET('python','sql','java') -- stores ZERO OR MORE values from a predefined list
```

RULES & NOTES

- ENUM uses 1 or 2 bytes depending on the number of members — this is commonly used for GENDER
ENUM('MALE','FEMALE').
- SET uses 1-8 bytes depending on the number of members. Unlike ENUM (pick exactly one), SET lets a single row store multiple tags at once — e.g. a student who knows both Python AND SQL.

SPECIAL DATATYPES

Datatype	Description
JSON	Stores JSON documents in MySQL's native JSON representation.
BOOLEAN / BOOL	A synonym for TINYINT(1); storage: 1 byte.

5.5 Constraints Recap

Definition: Constraints declared alongside a column/table definition to enforce data integrity. Fully explained in Module 4 — here they're shown in the context of CREATE TABLE.

EXAMPLE

```
-- STUDENTS: the parent table, STDID is its own PRIMARY KEY
CREATE TABLE STUDENTS(
  STDID INT PRIMARY KEY AUTO_INCREMENT,
  NAME VARCHAR(30) NOT NULL
);
-- MARKS: a separate child table — STDID here is a FOREIGN KEY pointing back to STUDENTS
CREATE TABLE MARKS(
  STDID INT,
  S1 INT CHECK(S1>=0),
  FOREIGN KEY (STDID) REFERENCES STUDENTS(STDID)
);
```

See Module 4 for full details.

5.5b Composite (Multi-Column) Primary Key

Definition: A PRIMARY KEY made up of two or more columns together, where the combination of their values must be unique (even if each individual column repeats on its own).

SYNTAX

```
CREATE TABLE table_name (  
    col1 datatype, col2 datatype, ...  
    PRIMARY KEY (col1, col2)  
);
```

RULES & NOTES

- No single column needs to be unique alone — only the pair/set together must be unique per row.
- Common in "linking" tables connecting two other tables.

EXAMPLE

```
CREATE TABLE orderdetails (  
    orderNumber int, productCode varchar(15) NOT NULL,  
    quantityOrdered int NOT NULL, priceEach decimal(10,2) NOT NULL,  
    orderLineNumber SMALLINT NOT NULL,  
    PRIMARY KEY (orderNumber, productCode)  
);
```

5.6 DESCRIBE / DESC

Definition: Shows the structure (columns, data types, keys, nullability, defaults) of a table.

SYNTAX

```
DESCRIBE table_name; -- or DESC table_name;
```

EXAMPLE

```
DESCRIBE TELUGU_MOVIES;  
DESC STUDENTS;
```

5.7 ALTER TABLE — ADD COLUMN

Definition: Adds one or more new columns to an existing table.

SYNTAX

```
ALTER TABLE table_name ADD COLUMN col_name datatype [constraint];  
-- multiple at once:  
ALTER TABLE table_name ADD COLUMN col1 type, ADD COLUMN col2 type;
```

RULES & NOTES

- Use AFTER col_name to control the position of the new column.

EXAMPLE

```
ALTER TABLE STUDENTS ADD COLUMN AGE TINYINT CHECK(AGE>5);  
ALTER TABLE STUDENTS ADD COLUMN LOCATION VARCHAR(50), ADD COLUMN ADDRESS VARCHAR(30);
```

5.8 ALTER TABLE — DROP COLUMN

Definition: Removes one or more existing columns from a table.

SYNTAX

```
ALTER TABLE table_name DROP COLUMN col_name;  
ALTER TABLE table_name DROP col1, DROP col2;
```

RULES & NOTES

- Permanently deletes the column and its data — cannot be undone once committed (DDL auto-commits).

EXAMPLE

```
ALTER TABLE STUDENTS DROP LOCATION, DROP ADDRESS;
```

5.9 ALTER TABLE — MODIFY COLUMN

Definition: Changes the data type, size, or constraints of an existing column (name stays the same).

SYNTAX

```
ALTER TABLE table_name MODIFY COLUMN col_name new_datatype [constraint];
```

EXAMPLE

```
ALTER TABLE STUDENTS MODIFY COLUMN GENDER ENUM('MALE','FEMALE');  
ALTER TABLE STUDENTS MODIFY COLUMN CLASS TINYINT UNSIGNED NOT NULL;
```

5.10 ALTER TABLE — RENAME COLUMN

Definition: Renames an existing column without changing its data or type.

SYNTAX

```
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
```

EXAMPLE

```
ALTER TABLE tips RENAME COLUMN sex TO gender;
```

5.11 Generated (Computed) Columns

Definition: A column whose value is automatically calculated from other columns in the same row.

SYNTAX

```
ALTER TABLE table_name ADD COLUMN new_col datatype GENERATED ALWAYS AS (expression) [STORED|VIRTUAL];
```

RULES & NOTES

- **VIRTUAL** (default) — computed on the fly when read, not physically stored.
- **STORED** — physically stored and updated whenever source columns change.
- Cannot directly INSERT/UPDATE a generated column's value — it is always derived.

EXAMPLE

```
ALTER TABLE marks ADD COLUMN percentage DECIMAL(5,2) GENERATED ALWAYS AS ((s1+s2+s3)/3);  
ALTER TABLE marks ADD COLUMN total_marks INT GENERATED ALWAYS AS (s1+s2+s3) AFTER s3;
```

5.12 DROP TABLE / DROP DATABASE

Definition: Permanently deletes an entire table (structure + data) or an entire database (all its tables).

SYNTAX

```
DROP TABLE [IF EXISTS] table_name;  
DROP DATABASE [IF EXISTS] database_name;
```

RULES & NOTES

- Irreversible — no WHERE clause; the whole object is removed.

EXAMPLE

```
DROP TABLE IF EXISTS old_backup;  
DROP DATABASE IF EXISTS test_db;
```

5.13 TRUNCATE TABLE

Definition: Removes ALL rows from a table in one fast operation while keeping the table structure intact — behaves like DDL (auto-commits) even though it deletes data.

SYNTAX

```
TRUNCATE TABLE table_name;
```

RULES & NOTES

- Cannot use a WHERE clause — it's all rows or nothing.
- Resets any AUTO_INCREMENT counter back to its starting value.
- Much faster than DELETE FROM table; for clearing an entire table. Full comparison in Module 7.4.

EXAMPLE

```
TRUNCATE TABLE students_backup;
```

5.14 CREATE TABLE ... LIKE

Definition: Creates a new, empty table based on the definition of an existing table, including its column attributes and indexes — but no data.

SYNTAX

```
CREATE TABLE new_table LIKE existing_table;
```

RULES & NOTES

- Unlike CREATE TABLE ... AS SELECT (CTAS, Module 6.4), this DOES preserve indexes, PRIMARY KEY, AUTO_INCREMENT, and CHECK constraints — but copies zero rows.
- **Important gotcha:** it does NOT preserve FOREIGN KEY definitions. If the source table has foreign keys, you must add them manually afterward with ALTER TABLE ... ADD FOREIGN KEY — this is a common real-world trap.
- To copy data in afterward: INSERT INTO new_table SELECT * FROM existing_table; (Module 6.5).

EXAMPLE

```
CREATE TABLE STUDENTS2 LIKE STUDENTS;
SELECT * FROM STUDENTS2;
-- If STUDENTS had a FOREIGN KEY, STUDENTS2 does NOT — add it separately if needed:
-- ALTER TABLE STUDENTS2 ADD FOREIGN KEY (STDID) REFERENCES SOME_OTHER_TABLE(ID);
```

DML — INSERT & UPDATE

Getting data into your tables, and safely modifying what's already there.

6.1 INSERT — Positional Values

Definition: Adds a new row by supplying values for every column in the table, in the exact order the columns were defined.

SYNTAX

```
INSERT INTO table_name VALUES (val1, val2, val3, ...);
```

RULES & NOTES

- You must provide a value for every column, in the table's defined order.
- Risky in real projects — adding/reordering columns later breaks this style of insert.

EXAMPLE

```
INSERT INTO students VALUES (101,"ravi","male",6,11,"2015-01-01");
```

6.2 INSERT with Identity (AUTO_INCREMENT)

Definition: When a column is defined with AUTO_INCREMENT, you can omit it from an INSERT and MySQL automatically generates the next available value for it.

SYNTAX

```
INSERT INTO table_name (col2, col3, ...) VALUES (val2, val3, ...);
-- id column (AUTO_INCREMENT) is skipped and filled automatically
SELECT LAST_INSERT_ID(); -- retrieves the last auto-generated id in this session
```

RULES & NOTES

- Only one AUTO_INCREMENT column is allowed per table, and it must be indexed — commonly as the PRIMARY KEY, though that's not strictly required.
- **Values are generally ascending, but not guaranteed gap-free or perfectly sequential** — failed inserts, rollbacks, deletions, and concurrent sessions can all create gaps. Never rely on AUTO_INCREMENT values being contiguous.
- You can also pass NULL explicitly in the identity column's position to trigger auto-generation.

EXAMPLE

```
INSERT INTO EMPLOYEES(NAME,DEPT,SALARY)
VALUES("RAVI","IT",50000),("KAVI","HR",40000);
-- ID column is AUTO_INCREMENT and is generated automatically for each row
```

6.3 Named Columns & Multi-Row Insert

Definition: Adds one or more rows by explicitly naming which columns receive values — safer and clearer than positional insert.

SYNTAX

```
INSERT INTO table_name (col1, col2, ...) VALUES
(val1, val2, ...),
(val1, val2, ...);
```

RULES & NOTES

- Columns not listed get their DEFAULT value or NULL (must be nullable).
- Multiple rows can be inserted in one statement — much faster than multiple single INSERTs.

EXAMPLE

```
INSERT INTO students (name,gender,class,age,date_of_birth) VALUES
("babu","male",7,12,"2014-01-01"),
("pavan","male",15,10,"2004-08-10");
```

6.4 INSERT: Creating a Table from Another Table (CTAS)

Definition: Creates a new table AND populates it in one step, using the result of a SELECT query against an existing table.

SYNTAX

```
CREATE TABLE new_table AS SELECT ... FROM old_table [WHERE ...];
```

RULES & NOTES

- Copies structure (basic column types) AND data, but NOT constraints like PRIMARY KEY, AUTO_INCREMENT, or FOREIGN KEY.
- WHERE 1=0 (always false) copies only the structure, with zero rows.

EXAMPLE

```
CREATE TABLE STUDENTS_INFO AS SELECT STDID,NAME,CLASS FROM STUDENTS;  
CREATE TABLE STUDENTS1 AS SELECT * FROM STUDENTS WHERE 1=0;
```

6.5 INSERT: Rows from One Table to Another

Definition: Copies existing rows from a source table straight into an already-existing target table, using a SELECT instead of literal VALUES.

SYNTAX

```
INSERT INTO target_table (col1, col2, ...)  
SELECT col1, col2, ... FROM source_table WHERE condition;
```

RULES & NOTES

- Unlike CTAS, the target table must already exist (e.g. created with CREATE TABLE ... LIKE, Module 5.14) with a matching column list.
- Great for populating a backup/archive table that keeps constraints intact.

EXAMPLE

```
CREATE TABLE STUDENTS2 LIKE STUDENTS;  
INSERT INTO STUDENTS2 SELECT * FROM STUDENTS WHERE CLASS = 6;
```

6.6 UPDATE

Definition: Modifies existing values in one or more columns for rows that match a condition.

SYNTAX

```
UPDATE table_name SET col1 = value1, col2 = value2 WHERE condition;
```

RULES & NOTES

- Always use WHERE — omitting it updates every row in the table.
- Multiple columns can be updated in a single SET, separated by commas.

EXAMPLE

```
UPDATE marks SET s1=s1-5 WHERE stdid=101;  
UPDATE marks SET s1=s1-5, s2=s2-5 WHERE stdid=102;
```

6.7 UPDATE with Computed Column

Definition: A column can reference itself (or other columns in the same row) in the new value expression, letting you update a value based on its current state rather than a fixed literal.

SYNTAX

```
UPDATE table_name SET column = column ± expression WHERE condition;
```

RULES & NOTES

- When a column appears on both sides (like amount = amount-2000), MySQL uses that column's *current* value on the right-hand side.
- For a single-table UPDATE with multiple assignments, MySQL generally evaluates them left-to-right — so a later

assignment in the same statement can see an earlier assignment's already-updated value.

- This is exactly how money moves between accounts using this pattern.

EXAMPLE

```
update accounts set amount = amount-2000 where acc_no=101;  
update accounts set amount = amount+2000 where acc_no=103;
```

ALTER TABLE (DDL) & DELETE (DML)

Removing rows safely, and the classic interview question that trips almost everyone up: DELETE vs TRUNCATE vs DROP.

7.1 ALTER: ADD Column

Definition: Adds a new column to an existing table (already introduced structurally in Module 5.7 — repeated here in the DML-adjacent context of evolving a table as data requirements change).

SYNTAX

```
ALTER TABLE table_name ADD COLUMN col_name datatype [constraint];
```

EXAMPLE

```
ALTER TABLE STUDENTS ADD COLUMN DATE_OF_BIRTH DATE;
```

7.2 MODIFY Column

Definition: Changes an existing column's data type/size/constraints without renaming it — often needed when inserted or updated data shows that the original column definition is too restrictive (e.g. a PERCENTAGE column overflowing).

SYNTAX

```
ALTER TABLE table_name MODIFY COLUMN col_name new_datatype [constraint];
```

EXAMPLE

```
alter table marks modify column percentage decimal(5,2);
alter table accounts modify column name varchar(40) not null, modify column amount bigint not null;
```

7.3 DELETE with WHERE

Definition: Removes existing rows from a table based on a condition.

SYNTAX

```
DELETE FROM table_name WHERE condition;
```

RULES & NOTES

- Without WHERE, DELETE removes all rows (but keeps table structure, unlike DROP TABLE).
- For a transactional storage engine such as InnoDB (the MySQL default), DELETE participates in transactions and its changes can be rolled back before COMMIT — unlike TRUNCATE, which auto-commits and can't be undone.

EXAMPLE

```
DELETE FROM students WHERE stdid = 111;
```

7.4 TRUNCATE vs DELETE — Difference

Definition: Three different ways to remove data/objects, often confused — this is one of the most common SQL interview questions.

SYNTAX

```
DELETE FROM t WHERE ...;  -- DML, row-by-row, can filter, can rollback
TRUNCATE TABLE t;        -- removes ALL rows at once, resets AUTO_INCREMENT, cannot filter
DROP TABLE t;            -- removes the table itself (structure + data)
```

Aspect	DELETE	TRUNCATE	DROP
Type	DML	DDL-like (auto-commits)	DDL
WHERE clause	Yes	No	No
Speed	Slower (each row change is logged individually)	Very fast	Very fast

Rollback (in transaction)	Yes	No (auto-commits)	No (auto-commits)
AUTO_INCREMENT counter	Unchanged	Reset	Table gone entirely
Table structure after	Kept	Kept	Removed

EXAMPLE

```
DELETE FROM students WHERE class = 6;  -- removes only class 6 students
TRUNCATE TABLE students_backup;      -- wipes ALL rows, keeps the table
DROP TABLE students_backup;          -- removes the table entirely
```

MODULE 8

DQL — SELECT Fundamentals

The core reading skill of SQL: pulling exactly the columns and rows you want, naming them clearly, sorting, deduplicating, and summarizing per group.

8.1 SELECT * vs Specific Columns

Definition: Retrieves all columns (*) or only the specified columns from a table.

SYNTAX

```
SELECT * FROM table_name;  
SELECT col1, col2 FROM table_name;
```

RULES & NOTES

- SELECT * is convenient but less efficient/readable in production — prefer naming columns explicitly.

EXAMPLE

```
SELECT * FROM CUSTOMERS;  
SELECT stdid, name FROM students WHERE class=6;
```

8.2 Column Aliasing (AS)

Definition: Gives a temporary display name to a column or expression in the result set.

SYNTAX

```
SELECT column AS alias_name FROM table_name;  
SELECT column 'Multi Word Alias' FROM table_name;
```

RULES & NOTES

- AS is optional — a space also works.
- Wrap a multi-word alias in quotes (or backticks) so MySQL treats it as one name, e.g. AS 'FULL NAME'; AS is recommended for readability even though it's optional.

EXAMPLE

```
SELECT CONCAT_WS(" ", CONTACTFIRSTNAME, CONTACTLASTNAME) AS 'FULL NAME' FROM CUSTOMERS;
```

8.3 WHERE Clause

Definition: Filters rows based on a condition before grouping/sorting happens.

SYNTAX

```
SELECT ... FROM table WHERE condition;
```

RULES & NOTES

- Can compare a column to a literal, or to another column (e.g. SHIPPEDDATE > REQUIREDDATE).
- Full operator reference (comparison, logical, arithmetic, etc.) is in Module 9.

EXAMPLE

```
SELECT * FROM ORDERS WHERE SHIPPEDDATE > REQUIREDDATE;
```

8.4 ORDER BY

Definition: Sorts the result set by one or more columns/expressions.

SYNTAX

```
SELECT ... FROM table ORDER BY column [ASC|DESC];
```

RULES & NOTES

- ASC (ascending) is the default; DESC is descending.

- Can sort by multiple columns: `ORDER BY col1, col2 DESC`.

EXAMPLE

```
SELECT * FROM tips WHERE gender="female" ORDER BY total_bill DESC;
```

8.5 DISTINCT Keyword

Definition: Removes duplicate rows from the result set, keeping only unique combinations of the selected columns.

SYNTAX

```
SELECT DISTINCT column FROM table_name;
```

RULES & NOTES

- Applies to the full combination of all selected columns, not just one, if multiple are listed.

EXAMPLE

```
SELECT DISTINCT(DAY) FROM TIPS;  
SELECT DISTINCT(GENDER) FROM TIPS;
```

8.6 GROUP BY with HAVING

Definition: GROUP BY groups rows sharing the same value(s) so aggregate functions can summarize each group; HAVING then filters those groups AFTER aggregation (whereas WHERE filters rows BEFORE aggregation).

SYNTAX

```
SELECT column, AGG_FUNC(column2)  
FROM table  
WHERE row_condition  
GROUP BY column  
HAVING group_condition;
```

RULES & NOTES

- Under MySQL's default ONLY_FULL_GROUP_BY SQL mode (on since 5.7.5), every non-aggregated column in SELECT must either appear in GROUP BY or be *functionally dependent* on the GROUP BY columns (e.g. selecting a customer's name is fine when you grouped by their unique customer number, since the number determines the name). This is a server setting, not an unconditional SQL rule — if disabled, MySQL still runs such queries but picks an arbitrary value per group.
- You cannot use an aggregate function inside WHERE — that's exactly what HAVING is for.

EXAMPLE

```
SELECT SIZE, COUNT(*) FROM TIPS GROUP BY SIZE HAVING COUNT(*) > 30;
```

8.7 Logical Order of Execution

Definition: The conceptual logical order used to understand how a SELECT statement's clauses relate to each other — different from the order you type them, and not necessarily MySQL's literal physical execution plan (the query optimizer can transform/reorder operations internally).

SYNTAX

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

RULES & NOTES

- This is why you can't use a SELECT alias inside WHERE, but you often can in ORDER BY/HAVING (MySQL-specific leniency).
- Understanding this order explains many "why doesn't my query work" errors.

MODULE 9

All Operators + DQL — Filtering & NULL Handling

Every category of operator MySQL supports, plus the filtering tools (BETWEEN, IN, LIKE, NULL checks) that turn a WHERE clause into a precise filter.

9.1 Arithmetic Operators

Definition: Perform mathematical calculations directly inside a SQL expression, on numeric column values or literals.

SYNTAX

```
+   addition
-   subtraction
*   multiplication
/   division
%   or MOD    - remainder (modulo)
```

RULES & NOTES

- Can be used in SELECT expressions, WHERE conditions, or SET clauses of UPDATE.
- Division / in MySQL returns a decimal result even for two integers (e.g. $5/2 = 2.5000$).

EXAMPLE

```
SELECT s1+s2+s3 AS total_marks FROM marks;
UPDATE accounts SET amount = amount - 2000 WHERE acc_no=101;
SELECT NUM % 2 FROM dual; -- remainder
```

9.2 Relational Operators

Definition: Compare two values and normally return TRUE (1) or FALSE (0) — the backbone of every WHERE condition. If either side of the comparison is NULL, the result is UNKNOWN (represented as NULL) rather than TRUE or FALSE — see Module 9.9 for why this matters.

SYNTAX

```
=   equal to
!=  or <>  not equal to
>   greater than
<   less than
>=  greater than or equal to
<=  less than or equal to
```

EXAMPLE

```
SELECT * FROM ORDERS WHERE SHIPPEDDATE>REQUIREDDATE;
select * from tips where size=2;
```

9.3 Logical Operators

Definition: Combine multiple conditions in a WHERE clause.

SYNTAX

```
WHERE cond1 AND cond2
WHERE cond1 OR cond2
WHERE NOT cond
```

RULES & NOTES

- AND requires all conditions true; OR requires at least one true.
- Use parentheses to control evaluation order when mixing AND/OR.

EXAMPLE

```
select * from tips where gender="female" and time="dinner";
select * from tips where size=2 or size=3;
```

9.4 Bitwise Operators

Definition: Operate on the individual binary bits of integer values — much rarer in everyday reporting queries but used for flags/permission masks.

SYNTAX

```
&   bitwise AND
|   bitwise OR
^   bitwise XOR
~   bitwise NOT (invert)
<<  left shift
>>  right shift
```

RULES & NOTES

- Common use case: a single INT column storing multiple on/off flags, checked with &.

EXAMPLE

```
SELECT 6 & 3;   -- 2   (0110 & 0011 = 0010)
SELECT 6 | 3;   -- 7   (0110 | 0011 = 0111)
SELECT 4 << 1;  -- 8
```

9.5 Assignment Operators

Definition: Assigns a value to a column (in UPDATE/SET) or to a user-defined session variable.

SYNTAX

```
SET @variable = value;
UPDATE table SET column = value WHERE condition;
SET @variable := value;  -- := also works, especially inside SELECT
```

RULES & NOTES

- Plain = works for assignment inside SET; use := if you need to assign within a SELECT list (since = there means comparison).

EXAMPLE

```
SET @count=0;
CALL get_class_students_count(10,@count);
```

9.6 BETWEEN

Definition: Checks whether a value lies within an inclusive range.

SYNTAX

```
SELECT ... WHERE column BETWEEN low AND high;
```

EXAMPLE

```
SELECT * FROM tips WHERE total_bill BETWEEN 10 AND 20;
```

9.7 IN

Definition: Checks whether a value matches any value in a given list.

SYNTAX

```
SELECT ... WHERE column IN (val1, val2, val3);
SELECT ... WHERE column NOT IN (val1, val2, val3);  -- negated: value matches NONE of the list
```

RULES & NOTES

- NOT IN is the direct negation of IN — returns rows whose value doesn't match any item in the list.
- Caution: if the list (often a subquery) can contain NULL, NOT IN can unexpectedly return zero rows — NULL comparisons are never true. NOT EXISTS (Module 15.6) is a safer alternative when NULLs are possible.

EXAMPLE

```
SELECT * FROM tips WHERE size IN (2,3,4);
SELECT * FROM tips WHERE size NOT IN (2,3,4); -- every size except 2, 3, or 4
```

9.8 LIKE, RLIKE

Definition: LIKE performs pattern matching using simple wildcard characters. RLIKE (a MySQL alias for REGEXP) performs pattern matching using full regular expressions — far more powerful for complex text patterns.

SYNTAX

```
SELECT ... WHERE column LIKE pattern;
-- % matches any sequence of characters (including none)
-- _ matches exactly one character
SELECT ... WHERE column RLIKE regex_pattern; -- regular expression matching
```

RULES & NOTES

- LIKE 'A%' — starts with A. LIKE '%A' — ends with A. LIKE '%A%' — contains A anywhere.
- NOT LIKE negates the pattern match.
- RLIKE/REGEXP supports full regex syntax — anchors (^, \$), character classes ([0-9]), alternation (|), etc. — useful when LIKE's two wildcards aren't expressive enough.

EXAMPLE

```
SELECT * FROM customers WHERE customername LIKE 'A%'; -- starts with A
SELECT * FROM customers WHERE contactlastname LIKE '_a%'; -- 2nd letter is 'a'
SELECT * FROM customers WHERE phone RLIKE '^([0-9]{3}-[0-9]{3})$'; -- regex: EXACTLY three digits, dash, three digits
-- (note the trailing $ anchor – without it, the pattern would also match longer strings
-- that merely START with that shape, e.g. "123-456-7890")
```

9.9 IS NULL / IS NOT NULL

Definition: Tests whether a column's value is missing (NULL) or present.

SYNTAX

```
SELECT ... WHERE column IS NULL;
SELECT ... WHERE column IS NOT NULL;
```

RULES & NOTES

- NULL cannot be tested with = or !=; you must use IS NULL / IS NOT NULL.

EXAMPLE

```
SELECT * FROM CUSTOMERS WHERE STATE IS NULL;
SELECT * FROM ORDERS WHERE COMMENTS IS NOT NULL;
```

9.10 ISNULL() Function

Definition: A function form of a NULL check — returns 1 if the expression is NULL, and 0 otherwise.

Important: ISNULL() in MySQL only ever takes ONE argument. (SQL Server has a different two-argument ISNULL(expr, replacement) that behaves like IFNULL — that is NOT how MySQL's version works, and mixing the two up is a common cross-database mistake.) If you want a "replace NULL with a default" function in MySQL, that's what IFNULL() / COALESCE() below are for.

SYNTAX

```
ISNULL(expression) -- returns 1 or 0
IFNULL(expression, default) -- returns 'default' if expression is NULL, else the expression itself
```

RULES & NOTES

- ISNULL(x) is a boolean check, functionally equivalent to x IS NULL but usable as an expression value (e.g. inside SELECT).
- IFNULL() is the more commonly used "replace NULL with a default" function — comparable to COALESCE().

EXAMPLE

```
SELECT customername, ISNULL(state) AS state_missing FROM customers;
SELECT customername, IFNULL(state, 'UNKNOWN') AS state FROM customers;
```

9.11 Column Aliases (AS) — Recap

Definition: Already introduced in Module 8.2 — repeated here alongside the other operators/predicates since aliasing is frequently combined with them in the same SELECT.

EXAMPLE

```
SELECT LOWER(COUNTRY) AS COUNTRY, COUNT(*) AS TOTAL_COUNT FROM CUSTOMERS GROUP BY COUNTRY;
```

9.12 Predicates Recap: BETWEEN, IN, LIKE, IS NULL

Definition: A predicate is any expression that evaluates to true/false/unknown inside a WHERE clause. BETWEEN, IN, LIKE, and IS NULL are the four most common special predicates beyond plain comparison operators — summarized together here for quick reference.

Predicate	Purpose	Example
BETWEEN	Value falls in an inclusive range	total_bill BETWEEN 10 AND 20
IN	Value matches any in a list	size IN (2,3,4)
LIKE	Value matches a text pattern	customername LIKE 'A%'
IS NULL	Value is missing	state IS NULL

9.13 EXISTS (Preview)

Definition: A special predicate that checks whether a subquery returns any rows at all. Introduced here as an operator category preview; fully explained with correlated examples in Module 15 (Subqueries).

SYNTAX

```
SELECT ... WHERE EXISTS (SELECT 1 FROM other_table WHERE condition);
```

MODULE 10

Numeric + String Functions

Built-in functions for calculating with numbers and for building/cleaning up text values.

10.1 Numeric Functions: ROUND, CEIL, FLOOR

Definition: Functions that adjust a decimal number to a target precision or to the nearest whole number.

SYNTAX

```
ROUND(number, decimals)  -- rounds to nearest, decimals optional (default 0)
CEIL(number)             -- rounds UP to the next whole number
FLOOR(number)            -- rounds DOWN to the previous whole number
```

EXAMPLE

```
SELECT ROUND(4.567, 2); -- 4.57
SELECT ROUND(10.256,2); -- 10.26
SELECT ROUND(10.254,2); -- 10.25
SELECT ROUND(10.9,0);   -- 11
SELECT CEIL(4.1);       -- 5
SELECT FLOOR(DATEDIFF(NOW(), '2004-08-10')/365) AS AGE;
```

10.1b TRUNCATE() (Numeric Function)

Definition: Cuts a decimal number down to a given number of decimal places *without rounding* — simply discards the extra digits.

SYNTAX

```
TRUNCATE(number, decimals)
```

RULES & NOTES

- Easy to confuse with TRUNCATE TABLE (Module 5.13) — they share a name but do completely different things. This TRUNCATE() is a numeric function that trims decimal digits; TRUNCATE TABLE deletes all rows from a table.
- Unlike ROUND(), this never rounds up — it always cuts straight down toward zero.

EXAMPLE

```
SELECT TRUNCATE(10.267, 2); -- 10.26 (just cuts, doesn't round to 10.27)
SELECT ROUND(10.267, 2);   -- 10.27 (rounds, for comparison)
```

10.2 POWER / POW, SQRT, ABS, MOD

Definition: Core mathematical functions for exponents, roots, absolute value, and remainders.

SYNTAX

```
POWER(base, exponent)
POW(base, exponent)  -- exact synonym for POWER() — both are correct, interchangeable
SQRT(number)
ABS(number)
MOD(number, divisor) -- equivalent to number % divisor
```

RULES & NOTES

- POW() and POWER() are two names for the exact same function in MySQL — use whichever reads better to you.

EXAMPLE

```
SELECT POWER(2,3); -- 8
SELECT POW(10,2);  -- 100 (same function, different name)
SELECT SQRT(49);   -- 7
SELECT SQRT(25);   -- 5
SELECT ABS(old.amount-new.amount); -- ABS() gets a positive transaction amount
SELECT MOD(10,3);  -- 1
```

10.3 RAND, SIGN, GREATEST, LEAST

Definition: Utility numeric functions — generating randomness, testing the sign of a number, and picking extremes across a set of expressions (row-wise, not column-wise like MAX/MIN).

SYNTAX

```
RAND()                -- random float between 0 and 1
SIGN(number)          -- returns -1, 0, or 1
GREATEST(val1, val2, ...)
LEAST(val1, val2, ...)
```

RULES & NOTES

- GREATEST/LEAST compare values *within the same row* (e.g. comparing s1, s2, s3 of one student), unlike MAX/MIN which aggregate *across rows* (Module 11.10).
- SIGN() returns +1 if the value is greater than 0, -1 if less than 0, and 0 if it equals 0.

EXAMPLE

```
SELECT SIGN(-5);  -- -1
SELECT SIGN(0);   -- 0
SELECT SIGN(5);   -- 1
SELECT GREATEST(s1,s2,s3) AS best_subject FROM marks;
SELECT LEAST(s1,s2,s3) AS worst_subject FROM marks;
```

10.4 String Functions: UPPER/UCASE, LOWER/LCASE

Definition: Converts text to all-uppercase or all-lowercase — useful for display formatting or case-insensitive grouping/comparison.

SYNTAX

```
UPPER(str)
LOWER(str)
UCASE(str)  -- exact synonym for UPPER() — both are correct, interchangeable
LCASE(str)  -- exact synonym for LOWER() — both are correct, interchangeable
```

RULES & NOTES

- UCASE/LCASE are older aliases kept for compatibility; UPPER/LOWER are the more commonly used names, but all four work identically in MySQL.

EXAMPLE

```
SELECT LOWER(COUNTRY) AS COUNTRY, COUNT(*) AS TOTAL_COUNT FROM CUSTOMERS GROUP BY COUNTRY;
SELECT UCASE(customername) FROM customers;  -- same result as UPPER(customername)
```

10.5 LENGTH, CHAR_LENGTH

Definition: Return the size of a string. LENGTH counts bytes; CHAR_LENGTH counts characters — they differ for multi-byte character sets (e.g. UTF-8 non-ASCII text).

SYNTAX

```
LENGTH(str)
CHAR_LENGTH(str)
```

EXAMPLE

```
SELECT customername, LENGTH(customername), CHAR_LENGTH(customername) FROM customers;
```

10.6 SUBSTRING, LEFT, RIGHT

Definition: Extract a portion of a string — from anywhere (SUBSTRING), from the start (LEFT), or from the end (RIGHT).

SYNTAX

```
SUBSTRING(str, start, length)
LEFT(str, n)
RIGHT(str, n)
```

EXAMPLE

```
SELECT SUBSTRING(contactlastname,1,3) FROM customers;  -- first 3 letters
SELECT LEFT(contactlastname,3) FROM customers;
SELECT RIGHT(customernumber,2) FROM customers;
```

10.7 CONCAT, CONCAT_WS

Definition: Built-in functions to build text values by joining multiple strings together.

SYNTAX

```
CONCAT(str1, str2, ...)
CONCAT_WS(separator, str1, str2, ...)
```

RULES & NOTES

- CONCAT joins strings directly with no separator.
- CONCAT_WS inserts a given separator between each string (and skips NULLs automatically).
- **Common gotcha:** CONCAT() returns NULL if ANY single argument is NULL — e.g. CONCAT(first_name, ' ', middle_name, ' ', last_name) silently returns NULL for the whole thing if middle_name is NULL. CONCAT_WS() doesn't have this problem (it just skips the NULL), which is often the safer choice.

EXAMPLE

```
SELECT CONCAT(CONTACTLASTNAME,CUSTOMERNUMBER,"@classicmodels.com") AS email FROM CUSTOMERS;
SELECT CONCAT_WS(" ",CONTACTFIRSTNAME,CONTACTLASTNAME) AS 'FULL NAME' FROM CUSTOMERS;
SELECT CONCAT('code',' ','gnan');           -- code gnan
SELECT CONCAT_WS('-', 'code','gnan');       -- code-gnan
```

10.8 REPLACE, TRIM, LTRIM, RTRIM

Definition: REPLACE substitutes all occurrences of a substring with another. TRIM removes leading AND trailing whitespace; LTRIM removes only leading (left-side) whitespace; RTRIM removes only trailing (right-side) whitespace.

SYNTAX

```
REPLACE(str, from_str, to_str)
TRIM([BOTH|LEADING|TRAILING] [remstr FROM] str)
LTRIM(str)  -- removes leading spaces only
RTRIM(str)  -- removes trailing spaces only
```

EXAMPLE

```
SELECT REPLACE(phone,'-','') FROM customers;  -- strip dashes from phone numbers
SELECT TRIM(' hello ');  -- 'hello' (both sides trimmed)
SELECT LTRIM(' hello ');  -- 'hello ' (only left side trimmed)
SELECT RTRIM(' hello ');  -- ' hello' (only right side trimmed)
```

10.9 LPAD, RPAD

Definition: Pad a string on the left or right with a repeated character until it reaches a target length — often used to zero-pad IDs for display.

SYNTAX

```
LPAD(str, length, pad_string)
RPAD(str, length, pad_string)
```

EXAMPLE

```
SELECT LPAD(customernumber,6,'0') FROM customers;  -- e.g. 000103
SELECT LPAD('456',4,'0');  -- 0456
SELECT RPAD('456',4,'2');  -- 4562
```

10.10 LOCATE, INSTR

Definition: Both return the position (1-based) of the first occurrence of a substring within a string, or 0 if not found. Both functions return the same position for the same inputs — the only difference is their argument order.

SYNTAX

```
LOCATE(substring, str [, start_position])  -- substring comes FIRST
INSTR(str, substring)                     -- string comes FIRST
```

RULES & NOTES

- LOCATE and INSTR return identical results for the same inputs — the only difference is which argument comes first. Both are correct; use whichever argument order you find more natural.
- LOCATE supports an optional third argument to start searching from a given position.

EXAMPLE

```
SELECT LOCATE('@', 'ravi101@classicmodels.com');  -- 8
SELECT INSTR('ravi101@classicmodels.com', '@');   -- 8 (same result, arguments swapped)
SELECT LOCATE('n', 'codegnan');                  -- 6
```

10.11 SUBSTRING_INDEX()

Definition: Returns the substring of a string before (or after) a given occurrence of a delimiter — extremely useful for splitting a delimited string (like an email address or a domain name) without writing a full parsing routine.

SYNTAX

```
SUBSTRING_INDEX(str, delimiter, count)
-- count positive -> everything BEFORE the Nth delimiter, counting from the LEFT
-- count negative -> everything AFTER the Nth delimiter, counting from the RIGHT
```

RULES & NOTES

- The count is cumulative from whichever end you count from — not just the single segment between two delimiters.
- Classic use case: splitting an email address at '@', or pulling the top-level domain off a URL.

EXAMPLE

```
SELECT SUBSTRING_INDEX('www.codegnan.com', '.', 1);  -- 'www'           (everything before the 1st dot)
SELECT SUBSTRING_INDEX('www.codegnan.com', '.', 2);  -- 'www.codegnan'       (everything before the 2nd dot)
SELECT SUBSTRING_INDEX('www.codegnan.com', '.', -1); -- 'com'                (everything after the last dot)
SELECT SUBSTRING_INDEX('www.codegnan.com', '.', -2); -- 'codegnan.com'       (everything after the 2nd-to-last dot)
-- Applied to a CONCAT email example:
SELECT SUBSTRING_INDEX(CONCAT(CONTACTLASTNAME, CUSTOMERNUMBER, "@classicmodels.com"), '@', 1) AS username
FROM CUSTOMERS;  -- extracts just the part before @
```

MODULE 11

Date/Time + Aggregate + System Functions

Working with dates and times, summarizing data with aggregates, and asking the server about itself.

11.1 NOW() and CURRENT_TIMESTAMP()

Definition: Returns the current date and time according to the current MySQL session time zone (not necessarily an immutable server-local clock — see the note below).

SYNTAX

```
SELECT NOW();
SELECT CURRENT_TIMESTAMP();
```

RULES & NOTES

- Both are functionally equivalent in MySQL and return YYYY-MM-DD HH:MM:SS.
- MySQL evaluates these using the **session time zone** (SET time_zone = ...), which is usually the server's time zone but can be changed per connection.

EXAMPLE

```
SELECT NOW();
SELECT CURRENT_TIMESTAMP();
```

11.2 CURDATE(), CURTIME()

Definition: Return only the current date, or only the current time, as opposed to NOW() which returns both together.

SYNTAX

```
CURDATE()    -- e.g. 2026-09-13
CURTIME()    -- e.g. 14:32:07
```

EXAMPLE

```
SELECT CURDATE();
SELECT CURTIME();
SELECT DATEDIFF(CURDATE(), orderdate) FROM orders;
```

11.3 YEAR() / MONTH() / DAY() Extraction

Definition: Extracts a specific part (year, month, or day number) from a date value — often used with GROUP BY.

SYNTAX

```
YEAR(date) | MONTH(date) | DAY(date)
```

RULES & NOTES

- Very useful for yearly/monthly reporting: GROUP BY YEAR(date_column).

EXAMPLE

```
SELECT YEAR(ORDERDATE), COUNT(*) FROM ORDERS WHERE STATUS="SHIPPED" GROUP BY YEAR(ORDERDATE);
```

11.3b HOUR(), MINUTE(), SECOND(), EXTRACT()

Definition: Extends YEAR/MONTH/DAY down to the time portion of a value — extracting the hour, minute, or second. EXTRACT() is a more general, standard-SQL form that can pull out any of these (and more) using a single unified syntax.

SYNTAX

```
HOUR(time)
MINUTE(time)
SECOND(time)
EXTRACT(unit FROM date)  -- unit: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, etc.
```

RULES & NOTES

- `EXTRACT()` can do everything `YEAR()`/`MONTH()`/`YEAR()` etc. do individually, plus compound units like `YEAR_MONTH` — useful when the unit itself is dynamic/parameterized.

EXAMPLE

```
SELECT HOUR(NOW());           -- e.g. 14
SELECT MINUTE(NOW());         -- e.g. 32
SELECT SECOND(NOW());         -- e.g. 7
SELECT EXTRACT(YEAR FROM ORDERDATE) FROM ORDERS;  -- same result as YEAR(ORDERDATE)
SELECT EXTRACT(HOUR FROM NOW());
```

11.4 DAYNAME()

Definition: Returns the name of the weekday (e.g. "Monday") for a given date.

SYNTAX

```
DAYNAME(date)
```

EXAMPLE

```
SELECT ORDERNUMBER, DAYNAME(ORDERDATE) AS order_weekday FROM ORDERS;
```

11.5 DATEDIFF()

Definition: Returns the number of days between two date values.

SYNTAX

```
DATEDIFF(date1, date2)  -- returns date1 - date2, in days
```

RULES & NOTES

- If `date1` is later than `date2`, result is positive; if earlier, result is negative.

EXAMPLE

```
SELECT ORDERNUMBER, DATEDIFF(SHIPPEDDATE, ORDERDATE) AS DAYS_TAKEN FROM ORDERS;
```

11.5b TIMEDIFF()

Definition: Like `DATEDIFF`, but for `TIME` or `DATETIME` values — returns the difference between two time values as a time value (HH:MM:SS), not a day count.

SYNTAX

```
TIMEDIFF(time1, time2)
```

RULES & NOTES

- Use `DATEDIFF()` (Module 11.5) when you only care about whole days; use `TIMEDIFF()` when you need hours/minutes/seconds precision.

EXAMPLE

```
SELECT TIMEDIFF('12:59:00', '11:00:00');  -- 01:59:00
```

11.6 DATE_ADD, DATE_SUB

Definition: Add or subtract a time interval (days, months, years, etc.) from a date value.

SYNTAX

```
DATE_ADD(date, INTERVAL n unit)
DATE_SUB(date, INTERVAL n unit)
-- unit: DAY, MONTH, YEAR, HOUR, MINUTE, SECOND
```

EXAMPLE

```
SELECT DATE_ADD(ORDERDATE, INTERVAL 7 DAY) AS expected_ship FROM ORDERS;
SELECT DATE_SUB(NOW(), INTERVAL 1 YEAR) AS one_year_ago;
```

11.7 FLOOR() with DATEDIFF() — Age Calculation

Definition: Combining DATEDIFF (days between two dates) with FLOOR (round down) and division by 365 approximates age in whole years.

SYNTAX

```
SELECT FLOOR(DATEDIFF(NOW(), birth_date) / 365) AS AGE;
```

RULES & NOTES

- This is an approximation (ignores leap years precisely) but common in practice/training exercises.

EXAMPLE

```
SELECT FLOOR(datediff(now(), '2004-08-10')/365) AS AGE;
```

11.8 DATE_FORMAT()

Definition: Formats a date/time value into a custom display string using format specifiers.

SYNTAX

```
DATE_FORMAT(date, format_string)
-- common specifiers: %d day, %m month, %Y year(4), %h hour(12h), %i minutes, %s seconds, %p AM/PM
```

EXAMPLE

```
SELECT DATE_FORMAT(NOW(), '%d-%m-%Y %h:%i:%s %p') AS Now;
```

11.9 STR_TO_DATE()

Definition: The reverse of DATE_FORMAT — parses a text string into a proper DATE/DATETIME value, given the format the text is in.

SYNTAX

```
STR_TO_DATE(str, format_string)
```

RULES & NOTES

- Essential when importing dates stored as text in a non-standard format (e.g. "10-08-2004").

EXAMPLE

```
SELECT STR_TO_DATE('10-08-2004', '%d-%m-%Y'); -- 2004-08-10
```

11.10 Aggregate Functions: COUNT, SUM, MIN, MAX, AVG

Definition: Functions that summarize a set of rows into a single value — one value for the entire result set, or **one value per group** when combined with GROUP BY (Module 8.6).

SYNTAX

```
COUNT(*) | COUNT(column) | SUM(column) | MIN(column) | MAX(column) | AVG(column)
```

RULES & NOTES

- COUNT(*) counts all rows; COUNT(column) counts only non-NULL values in that column.
- Typically used alone (whole table) or combined with GROUP BY (per group, Module 8.6).

EXAMPLE

```
SELECT COUNT(*) FROM TIPS WHERE GENDER="MALE" AND SMOKER="YES";
SELECT MIN(TOTAL_BILL), MAX(TOTAL_BILL) FROM TIPS;
```

11.11 System Functions: VERSION(), DATABASE()

Definition: Functions that return information about the MySQL server or the current session, rather than about your data.

SYNTAX

```
VERSION()      -- MySQL server version string
DATABASE()     -- name of the currently active database
```

EXAMPLE

```
SELECT VERSION();      -- e.g. '8.0.36'
SELECT DATABASE();     -- e.g. 'classicmodels' (after USE classicmodels;)
```

11.12 System Functions: USER(), CONNECTION_ID()

Definition: USER() returns the current logged-in MySQL account (user@host); CONNECTION_ID() returns the unique numeric ID of the current client connection/session.

SYNTAX

```
USER()
CONNECTION_ID()
```

EXAMPLE

```
SELECT USER();          -- e.g. 'root@localhost'
SELECT CONNECTION_ID(); -- e.g. 42
```

11.13 System Functions: LAST_INSERT_ID()

Definition: Returns the AUTO_INCREMENT value generated by the most recent successful INSERT statement in the current session.

SYNTAX

```
SELECT LAST_INSERT_ID();
```

RULES & NOTES

- Session-specific — safe to use even with concurrent users, since it only reflects your own last insert.
- **Important gotcha for multi-row INSERT:** if a single INSERT statement adds several rows at once, LAST_INSERT_ID() returns the value generated for the **first** row of that statement, not the last — despite what the function's name suggests. This trips up a lot of people.

EXAMPLE

```
INSERT INTO EMPLOYEES (NAME, DEPT, SALARY) VALUES ("NEW HIRE", "IT", 50000);
SELECT LAST_INSERT_ID(); -- returns the auto-generated ID column value for that row
-- Multi-row case:
INSERT INTO EMPLOYEES (NAME, DEPT, SALARY) VALUES ("A", "IT", 1), ("B", "IT", 1), ("C", "IT", 1);
SELECT LAST_INSERT_ID(); -- returns the ID generated for "A" (the FIRST row), not "C"
```

Normalization (1NF, 2NF, 3NF)

The design discipline that explains WHY databases like classicmodels, school, and bank are split into separate related tables instead of one giant flat table — placed here because you need it before Joins make full sense, and it feeds directly into ER Modelling (Module 18).

12.1 Why Normalize?

Definition: Normalization is the step-by-step process of organizing columns and tables to minimize data redundancy and avoid update/insert/delete anomalies, by progressively applying a series of rules called "normal forms."

RULES & NOTES

- Each normal form (1NF, 2NF, 3NF, ...) fixes a specific category of redundancy problem. This guide covers 1NF through 3NF — the level needed for almost all everyday relational design.
- Normalization directly explains why databases like classicmodels and school are split into multiple linked tables (CUSTOMERS + ORDERS + ORDERDETAILS, or STUDENTS + MARKS) instead of one wide table.

EXAMPLE — AN UNNORMALIZED STARTING POINT

```
-- Imagine all order data crammed into ONE flat table instead of the actual
-- CUSTOMERS / ORDERS / ORDERDETAILS split:
ORDERS_FLAT(
  orderNumber, orderDate,
  customerNumber, customerName, customerCity,
  productCode1, productName1, quantity1,
  productCode2, productName2, quantity2
)
-- Problems: customerName repeats on every order row (redundancy),
-- and the number of products per order is hard-capped by column count.
```

12.2 Insertion, Update, and Deletion Anomalies

Definition: An **anomaly** is an unwanted side-effect caused by a badly designed (unnormalized) table — where the same fact is stored in more than one place. Three anomaly categories are commonly taught — insertion, update, and deletion — and normalization aims to eliminate them along with reducing unnecessary redundancy generally.

RULES & NOTES — ONE FLAT TABLE USED FOR ALL THREE EXAMPLES

```
ORDERDETAILS_FLAT(orderNumber, productCode, productName, quantityOrdered)
```

- **Insertion Anomaly** — you CANNOT insert one piece of information without also being forced to insert another, unrelated piece. Example: you can't add a brand-new product ("S99_9999", "New Widget") into the catalog until it's actually part of an order — because productName only exists inside an order row. There's no order-free place to just register a product.
- **Update Anomaly** — the same fact is duplicated across many rows, so updating it in one place but missing another leaves the data contradicting itself. Example: if "S10_1678" appears on 50 different order rows and its productName is misspelled, you'd have to update all 50 rows — miss even one, and now the same product has two different names in the same table.
- **Deletion Anomaly** — deleting one fact accidentally destroys a completely different, still-needed fact, because they were never stored separately. Example: if order 101 is the ONLY order that ever contained product "S12_1099", deleting order 101 wipes out all knowledge that "S12_1099" (and its name) ever existed — even though the product itself still exists.

Each of the three normal forms below removes exactly one of these problems — 1NF removes the structural mess that makes anomalies possible, 2NF removes partial-dependency duplication, and 3NF removes transitive-dependency duplication.

12.3 First Normal Form (1NF)

Definition: A table is in 1NF if every column holds a single, atomic (indivisible) value, and there are no repeating groups of columns (like productCode1, productCode2, ...) for the same kind of data.

RULES & NOTES

- Each cell must contain one value only — not a list, not multiple comma-separated values.
- No repeating column groups — instead, repeating data becomes its own set of rows in a related table.

- A primary key isn't technically part of the formal 1NF definition itself, but it's such standard good-design practice alongside it that every table in this guide has one — so every row can actually be told apart.

EXAMPLE — APPLYING 1NF

```
-- VIOLATES 1NF (multiple products crammed into one row/column):
ORDERS_FLAT(orderNumber, customerNumber, productCodes)
101, 103, "S10_1678, S10_4962, S12_1099" -- not atomic!
-- FIXED (1NF) — one row per order-product line, which is exactly the real table:
ORDERDETAILS(orderNumber, productCode, quantityOrdered, priceEach, orderLineNumber)
101, S10_1678, 30, 95.70, 1
101, S10_4962, 50, 43.13, 2
```

Anomaly this fixes: with products crammed into one comma-separated cell, you can't add/remove a single product from an order with a normal UPDATE — you'd have to rewrite the whole string. 1NF turns each product into its own row, so adding, updating, or removing one product line is a simple, isolated operation.

12.4 Second Normal Form (2NF)

Definition: A table is in 2NF if it is already in 1NF, AND every non-key column depends on the WHOLE primary key — not just part of it. This rule only matters when a table has a composite (multi-column) primary key.

RULES & NOTES

- A **partial dependency** is when a column depends on only one part of a composite key, not the full key together — this violates 2NF.
- Fix: move the partially-dependent column into its own table, keyed by the column it actually depends on.

EXAMPLE — APPLYING 2NF

```
-- VIOLATES 2NF: composite key (orderNumber, productCode), but productName
-- only depends on productCode alone (not on orderNumber too) — a partial dependency:
ORDERDETAILS_BAD(orderNumber, productCode, productName, quantityOrdered)
-- FIXED (2NF) — split productName out into its own table keyed by productCode,
-- exactly matching a real classicmodels design:
PRODUCTS(productCode, productName, ...)
ORDERDETAILS(orderNumber, productCode, quantityOrdered, priceEach)
-- now every non-key column in ORDERDETAILS depends on the FULL (orderNumber, productCode) pair
```

Anomalies this fixes: in ORDERDETAILS_BAD, productName repeats on every order line that includes that product — that's the **update anomaly** (fix a typo in one row, forget the other 49, now they disagree). It's also the **insertion anomaly** from 12.2 (can't register a new product with no order yet) and the **deletion anomaly** (delete the one order that used a product, and its name is gone forever). Moving productName into its own PRODUCTS table — keyed only by productCode — fixes all three at once: the name is stored exactly once, independently of any order.

12.5 Third Normal Form (3NF) — Practical Normalization

Definition: A table is in 3NF if it is already in 2NF, AND no non-key column depends on another non-key column (no "transitive dependency") — every non-key column must depend directly on the primary key, and nothing else.

RULES & NOTES

- A **transitive dependency**: Column C depends on Column B, and Column B depends on the key — so C only depends on the key *through* B.
- Fix: move the transitively-dependent column into its own table, keyed by the column it actually depends on.

EXAMPLE — APPLYING 3NF

```
-- VIOLATES 3NF: CLASS_TEACHER depends on CLASS, and CLASS depends on STDID —
-- so CLASS_TEACHER only depends on STDID transitively, through CLASS:
STUDENTS_BAD(STDID, NAME, CLASS, CLASS_TEACHER)
-- FIXED (3NF) — move CLASS_TEACHER into its own CLASSES table:
STUDENTS(STDID, NAME, CLASS) -- CLASS is now a foreign key
CLASSES(CLASS, CLASS_TEACHER) -- CLASS_TEACHER depends only on CLASS
```

Anomalies this fixes: in STUDENTS_BAD, every student in Class 6 repeats "Mrs. Rao" (say) in CLASS_TEACHER — an **update anomaly** waiting to happen (change the teacher for Class 6, but only update some students' rows). It's a **deletion anomaly** too: if Class 6 only has one student left and that student is deleted, you've lost the fact that Class 6's teacher is Mrs. Rao. And it's an **insertion anomaly**: you can't record "Class 9's teacher is Mr. Iyer" until at least one student is enrolled in Class 9. Moving CLASS_TEACHER into its own CLASSES table fixes all three.

EXAMPLE — 3NF APPLIED TO CLASSICMODELS (ORDERS)

```
-- VIOLATES 3NF: customerName/customerCity depend on customerNumber,  
-- not directly on orderNumber (the ORDERS primary key) – transitive:  
ORDERS_BAD(orderNumber, orderDate, customerNumber, customerName, customerCity)  
-- FIXED (3NF) – a real-world design:  
CUSTOMERS(customerNumber, customerName, city, ...)  
ORDERS(orderNumber, orderDate, customerNumber) -- customerNumber is a FOREIGN KEY
```

Once a design satisfies 1NF, 2NF, and 3NF, it typically maps directly onto the ER diagram you'll build in Module 18 — each 3NF table becomes one entity, and every FOREIGN KEY becomes a relationship line.

Set Operators + Joins Part 1

Two different ways to combine query results: stacking rows vertically (set operators) versus combining tables side-by-side on a matching key (joins).

13.1 UNION

Definition: Combines rows from two or more SELECT queries into a single result set, removing duplicate rows.

SYNTAX

```
(SELECT col FROM t1) UNION (SELECT col FROM t2);
```

RULES & NOTES

- Each SELECT must return the same number of columns with compatible data types.
- Automatically removes duplicate rows across the combined results.

EXAMPLE

```
(SELECT customernumber FROM customers) UNION (SELECT customernumber FROM orders);
```

13.2 UNION ALL

Definition: Same as UNION but keeps all duplicate rows instead of removing them.

SYNTAX

```
(SELECT col FROM t1) UNION ALL (SELECT col FROM t2);
```

RULES & NOTES

- Faster than UNION since MySQL skips the duplicate-removal step.

EXAMPLE

```
(SELECT customernumber FROM customers) UNION ALL (SELECT customernumber FROM orders);
```

13.3 INTERSECT

Definition: Returns only the rows that appear in BOTH result sets (common records).

SYNTAX

```
(SELECT col FROM t1) INTERSECT (SELECT col FROM t2);
```

RULES & NOTES

- Supported from MySQL 8.0.31 onward.
- Column count and compatible types required, same as UNION.

EXAMPLE

```
(select customernumber from customers) intersect (select customernumber from orders);
```

13.4 EXCEPT (a.k.a. MINUS in some databases)

Definition: Returns rows from the FIRST query that do NOT appear in the second query's results.

SYNTAX

```
(SELECT col FROM t1) EXCEPT (SELECT col FROM t2);
```

RULES & NOTES

- Order matters: (A EXCEPT B) is not the same as (B EXCEPT A).
- Supported from MySQL 8.0.31 onward.

EXAMPLE

```
(select customernumber from customers) except (select customernumber from orders);  
-- customers who never placed an order
```

13.5 INNER JOIN

Definition: Returns only the rows where there is a match in BOTH tables.

SYNTAX

```
SELECT ... FROM t1  
INNER JOIN t2 ON t1.col = t2.col;
```

RULES & NOTES

- Rows with no match on either side are excluded from the result.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER,CUSTOMERNAME,ORDERNUMBER FROM CUSTOMERS C  
INNER JOIN ORDERS O ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER;
```

13.6 EQUI JOIN (Old-Style Comma Join)

Definition: An "equi join" is any join whose condition uses the equality operator (=) — this includes a modern INNER JOIN ... ON a.x = b.x just as much as the older comma-style syntax below. The "comma join" is simply an older way of *writing* an equi join, not a different kind of join.

SYNTAX

```
SELECT ... FROM t1, t2 WHERE t1.col = t2.col; -- old-style comma syntax for an equi join
```

RULES & NOTES

- Functionally equivalent to INNER JOIN, but modern style (explicit JOIN...ON) is preferred for readability and to avoid accidental cross joins.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER,CUSTOMERNAME,ORDERNUMBER FROM CUSTOMERS C,ORDERS O  
WHERE C.CUSTOMERNUMBER = O.CUSTOMERNUMBER AND O.STATUS = "CANCELLED";
```

13.7 NATURAL JOIN

Definition: Automatically joins two tables using all columns that share the same name in both tables — no ON clause needed.

SYNTAX

```
SELECT ... FROM t1 NATURAL JOIN t2;
```

RULES & NOTES

- Risky: if the tables happen to share an unintended common column name, results can silently be wrong.
- Explicit INNER JOIN...ON is generally safer and preferred in real projects.

EXAMPLE

```
SELECT DISTINCT C.CUSTOMERNUMBER,CUSTOMERNAME FROM CUSTOMERS C  
NATURAL JOIN ORDERS O;
```

13.8 NON-EQUI JOIN

Definition: A join whose condition uses a comparison operator OTHER than equals (>, <, BETWEEN, etc.) instead of matching on equal key values.

SYNTAX

```
SELECT ... FROM t1 JOIN t2 ON t1.col > t2.col;
```

RULES & NOTES

- Common use case: matching a value to the correct "band" or "range" in a reference table (e.g. matching an order's total

to a discount-tier table).

EXAMPLE

```
-- Match each order to a shipping-fee tier based on amount ranges
SELECT o.orderNumber, t.tier_name
FROM orders o JOIN shipping_tiers t
ON o.total_amount BETWEEN t.min_amount AND t.max_amount;
```

Joins Part 2 (Outer & Cross Joins)

Joins that keep unmatched rows on purpose, the all-pairs Cartesian join, comparing a table to itself, and chaining several joins together.

14.1 LEFT JOIN (LEFT OUTER JOIN)

Definition: Returns ALL rows from the left table, plus matching rows from the right table. Unmatched right-side columns show as NULL.

SYNTAX

```
SELECT ... FROM t1
LEFT JOIN t2 ON t1.col = t2.col;
```

RULES & NOTES

- Useful for finding "all X, whether or not they have a related Y" — e.g. all customers, even those with zero orders.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER, C.CUSTOMERNAME, COUNT(ORDERNUMBER)
FROM CUSTOMERS C LEFT JOIN ORDERS O
ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER
GROUP BY C.CUSTOMERNUMBER;
```

14.2 RIGHT JOIN (RIGHT OUTER JOIN)

Definition: Returns ALL rows from the right table, plus matching rows from the left table. Unmatched left-side columns show as NULL.

SYNTAX

```
SELECT ... FROM t1
RIGHT JOIN t2 ON t1.col = t2.col;
```

RULES & NOTES

- Mirror image of LEFT JOIN — can always be rewritten as a LEFT JOIN by swapping table order.

EXAMPLE

```
SELECT O.ORDERNUMBER, C.CUSTOMERNAME
FROM CUSTOMERS C RIGHT JOIN ORDERS O
ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER;
```

14.3 FULL OUTER JOIN (MySQL Workaround)

Definition: Would return all rows from both tables (matched + unmatched from both sides). MySQL does NOT support FULL OUTER JOIN natively — it must be simulated.

SYNTAX

```
SELECT ... FROM t1 LEFT JOIN t2 ON t1.col = t2.col
UNION ALL
SELECT ... FROM t1 RIGHT JOIN t2 ON t1.col = t2.col
WHERE t1.col IS NULL; -- keep ONLY the right-side rows that had no left-side match
```

RULES & NOTES

- The LEFT JOIN already covers every matched row plus every unmatched left-side row. The second query then adds ONLY the unmatched right-side rows — filtered with WHERE t1.col IS NULL — so nothing gets double-counted.
- Use UNION ALL here, not plain UNION: since the WHERE filter already prevents overlap between the two halves, there's no need to pay for UNION's duplicate-removal pass — and a plain UNION could incorrectly collapse two genuinely identical (but distinct) result rows into one.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER, O.ORDERNUMBER FROM CUSTOMERS C LEFT JOIN ORDERS O ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER
UNION ALL
SELECT C.CUSTOMERNUMBER, O.ORDERNUMBER FROM CUSTOMERS C RIGHT JOIN ORDERS O ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER
WHERE C.CUSTOMERNUMBER IS NULL;
```

14.4 CROSS JOIN

Definition: Returns the Cartesian product — every row of table 1 combined with every row of table 2.

SYNTAX

```
SELECT ... FROM t1 CROSS JOIN t2;
```

RULES & NOTES

- No join condition is used.
- Result size = (rows in t1) x (rows in t2) — can get very large very fast.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER, CUSTOMERNAME, EMPLOYEENUMBER FROM CUSTOMERS C
CROSS JOIN EMPLOYEES;
```

14.5 JOIN with WHERE Filters & Anti-Join Patterns

Definition: An "anti-join" finds rows in one table that have NO matching row in another — typically written as a LEFT JOIN combined with a WHERE clause that filters for the NULL side.

SYNTAX

```
SELECT t1.* FROM t1
LEFT JOIN t2 ON t1.col = t2.col
WHERE t2.col IS NULL;
```

RULES & NOTES

- This can produce the same result as an EXCEPT query (Module 13.4) or a NOT EXISTS subquery (Module 15.6) when the join key and selected columns are chosen appropriately — though they aren't universally interchangeable: EXCEPT has set semantics (duplicates removed), while a JOIN can preserve row multiplicity.

EXAMPLE

```
-- Customers who have never placed an order (anti-join version of the Module 13.4 EXCEPT example)
SELECT C.CUSTOMERNUMBER, C.CUSTOMERNAME
FROM CUSTOMERS C LEFT JOIN ORDERS O ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER
WHERE O.CUSTOMERNUMBER IS NULL;
```

14.6 SELF JOIN

Definition: Joins a table to itself, typically to compare rows within the same table (e.g. an employee and their manager, both stored in the same EMPLOYEES table).

SYNTAX

```
SELECT ... FROM t1 A INNER JOIN t1 B ON A.col = B.other_col;
```

RULES & NOTES

- Requires table aliases (e.g. E and M) since the table name would otherwise be ambiguous.

EXAMPLE

```
SELECT E.EMPLOYEENUMBER,
CONCAT_WS(" ", E.LASTNAME, E.FIRSTNAME) AS 'EMPLOYEE NAME', E.REPORTSTO, CONCAT_WS(" ", M.LASTNAME, M.FIRSTNAME) AS 'MANAGER
NAME'
FROM EMPLOYEES E INNER JOIN EMPLOYEES M
ON E.REPORTSTO=M.EMPLOYEENUMBER;
```

14.7 Multi-Table Joins

Definition: Chaining more than one JOIN in a single query to pull data from 3+ related tables.

SYNTAX

```
SELECT ... FROM t1
JOIN t2 ON t1.col = t2.col
JOIN t3 ON t2.col2 = t3.col2;
```

RULES & NOTES

- Each JOIN adds one more table; conditions can mix INNER/LEFT/RIGHT as needed.

EXAMPLE

```
SELECT C.CUSTOMERNUMBER , C.CUSTOMERNAME , O.ORDERNUMBER , OD.PRODUCTCODE
FROM CUSTOMERS C RIGHT JOIN ORDERS O
ON C.CUSTOMERNUMBER=O.CUSTOMERNUMBER
INNER JOIN ORDERDETAILS OD
ON OD.ORDERNUMBER=O.ORDERNUMBER;
```

Subqueries

Nesting one SELECT inside another so the result of one query drives the filtering or output of another.

15.1 Single-Row Subquery

Definition: A subquery that returns exactly one row and one column — usable directly with =, >, <, etc.

SYNTAX

```
SELECT ... FROM t WHERE column = (SELECT AGG_FUNC(col) FROM t);
```

RULES & NOTES

- If the subquery unexpectedly returns more than one row, MySQL raises an error with =, >, < operators.

EXAMPLE

```
SELECT * FROM EMPLOYEES WHERE SALARY > (SELECT AVG(SALARY) FROM EMPLOYEES);
```

15.2 Multi-Row Subquery (IN)

Definition: A subquery that can return multiple rows; used with IN to check membership.

SYNTAX

```
SELECT ... FROM t WHERE column IN (SELECT column FROM t2 WHERE ...);
```

RULES & NOTES

- IN checks if the outer value matches ANY value returned by the subquery.

EXAMPLE

```
SELECT * FROM EMPLOYEES WHERE DEPT IN (SELECT DEPT FROM EMPLOYEES WHERE NAME IN ("KAVI", "KISHORE"));
```

15.3 ANY / SOME / ALL Nested Subqueries

Definition: ANY (and its exact synonym SOME) compares a value against each value returned by a subquery — true if it matches the condition against at least one; ALL requires it to satisfy the condition against every value.

SYNTAX

```
SELECT ... WHERE column > ANY (subquery);
SELECT ... WHERE column > SOME (subquery);  -- identical meaning to ANY
SELECT ... WHERE column > ALL (subquery);
```

RULES & NOTES

- > ANY is TRUE when the comparison is TRUE for at least one non-NULL value returned by the subquery — informally, "greater than at least the smallest value returned."
- > ALL is TRUE only when the comparison is TRUE for every returned value — informally, "greater than even the largest value returned." Two easy-to-miss edge cases: if the subquery returns an **empty result**, > ALL is TRUE (vacuously — there's no value to fail the test) while > ANY is FALSE; and if the subquery's results include a NULL, the comparison can come out UNKNOWN rather than a clean TRUE/FALSE.
- SOME is pure syntax sugar for ANY — MySQL treats them identically.

EXAMPLE

```
SELECT * FROM EMPLOYEES WHERE SALARY > ANY(SELECT SALARY FROM EMPLOYEES WHERE DEPT = "SALES");
SELECT * FROM EMPLOYEES WHERE SALARY > ALL(SELECT SALARY FROM EMPLOYEES WHERE DEPT = "HR");
```

15.4 Subquery in FROM Clause (Derived Table)

Definition: A subquery used as if it were a temporary table, placed inside the FROM clause and given an alias.

SYNTAX

```
SELECT * FROM (SELECT ... FROM t GROUP BY ...) AS alias_name WHERE ...;
```

RULES & NOTES

- Must always be given an alias.
- Lets you filter/query on top of an already-aggregated result (e.g. filtering on a computed COUNT).

EXAMPLE

```
SELECT * FROM  
(SELECT DEPT, COUNT(*) AS TOTALCOUNT FROM EMPLOYEES GROUP BY DEPT) AS TABLEE  
WHERE TOTALCOUNT > 3;
```

15.5 Correlated Subquery

Definition: A subquery that references a column from the OUTER query, so its result depends on the current outer row. Conceptually, it's evaluated once per outer row (rather than once total) — though MySQL's optimizer may transform how it's actually executed internally.

SYNTAX

```
SELECT * FROM t1 E1  
WHERE column > (SELECT AGG_FUNC(column) FROM t1 E2 WHERE E1.group_col = E2.group_col);
```

RULES & NOTES

- Both the outer and inner query reference the SAME table, aliased differently (E1, E2).
- Classic use case: comparing each row to an aggregate of its own group (e.g. above department average, or the max/min within each department).

EXAMPLE

```
SELECT * FROM EMPLOYEES E1  
WHERE SALARY = (SELECT MAX(SALARY) FROM EMPLOYEES E2 WHERE E1.DEPT = E2.DEPT);
```

15.5b Nested Subquery

Definition: A subquery placed inside ANOTHER subquery — i.e. a subquery nested two (or more) levels deep, rather than just one level inside the outer query.

SYNTAX

```
SELECT * FROM t1  
WHERE col IN (  
  SELECT col FROM t2  
  WHERE col2 IN (  
    SELECT col2 FROM t3 WHERE condition  
  )  
);
```

RULES & NOTES

- This is a naming category based on *structure* (how many levels deep), whereas single-row/multi-row/correlated (Module 15.1-15.5) are categories based on *behavior*. A nested subquery can itself be single-row, multi-row, or correlated at each level.
- For learning purposes, nested subqueries can be understood from the innermost query outward — but MySQL's optimizer may transform the actual execution plan rather than literally running them in that order.

EXAMPLE

```
-- Employees in the same department as whoever earns the single highest salary company-wide  
SELECT * FROM EMPLOYEES  
WHERE DEPT = (  
  SELECT DEPT FROM EMPLOYEES  
  WHERE SALARY = (SELECT MAX(SALARY) FROM EMPLOYEES)  
);
```

15.6 EXISTS and NOT EXISTS

Definition: Checks whether a subquery returns ANY rows at all (true/false), rather than comparing actual values.

SYNTAX

```
SELECT ... FROM t1 WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.col = t1.col);
```

RULES & NOTES

- EXISTS can stop looking as soon as it finds one matching row, so it's often described as efficient for large subqueries — but don't assume it's always faster than IN; MySQL's optimizer can transform either form, and actual performance depends on indexes and data size.
- NOT EXISTS is commonly used to find "parent rows with no matching child rows" — the subquery equivalent of the EXCEPT set operator (Module 13.4) and the anti-join pattern (Module 14.5).

EXAMPLE

```
SELECT * FROM customers c  
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customernumber = c.customernumber);
```

Views + Transaction Management

Saving reusable queries as virtual tables, and controlling groups of DML statements so they succeed or fail together — the two features that make multi-user data changes safe.

16.1 CREATE VIEW

Definition: A view is a virtual table based on the result of a stored SELECT query. It stores the query definition rather than a persistent copy of the result — MySQL decides how to process it when queried (commonly merging it into the outer query, or in some cases materializing it as a temporary table), but conceptually it behaves as if it re-runs the underlying query each time it's accessed.

SYNTAX

```
CREATE VIEW view_name AS
SELECT ... FROM ...;
```

RULES & NOTES

- A view can be based on a simple SELECT, a JOIN, or even a subquery.
- Querying a view is identical syntax to querying a real table: `SELECT * FROM view_name`.

EXAMPLE

```
create view order_based_customer_details as select * from customers where customernumber in (select customernumber
from orders);
```

16.1b Advantages & Disadvantages of Views

Why you'd reach for a view instead of just writing the SELECT every time — and the one real cost to keep in mind.

ADVANTAGES

- **Code reusability** — write the JOIN/filter logic once (like the `order_based_customer_details` view shown earlier), then reuse it everywhere with a simple `SELECT * FROM view_name` instead of retyping the whole query.
- **Stores/abstracts business logic** — the complexity of the underlying query lives in one place, not duplicated across every application that needs it.
- **Hides the original table structure (data abstraction)** — consumers of the view don't need to know how many real tables are joined underneath, or what the base tables' exact columns are.
- **Security** — a view can expose only specific columns/rows of a sensitive base table, restricting what a user sees without touching the underlying table's permissions directly.

DISADVANTAGE

- A view isn't free performance-wise — depending on how MySQL processes it (Module 16.1), a view built on a heavy JOIN or aggregation can cost real memory/CPU on every query against it, similar to running that query directly.

16.2 CREATE OR REPLACE VIEW

Definition: Creates a view, or if a view with that name already exists, replaces its definition — avoiding a "view already exists" error.

SYNTAX

```
CREATE OR REPLACE VIEW view_name AS
SELECT ...;
```

RULES & NOTES

- Safer than `DROP VIEW + CREATE VIEW` when iterating on a view's definition during development.

EXAMPLE

```
CREATE OR REPLACE VIEW student_details AS SELECT stdid, name, class FROM students;
```

16.3 Querying & Describing a View

Definition: Once created, a view behaves like a read table for SELECT and DESCRIBE purposes.

SYNTAX

```
SELECT * FROM view_name;  
DESC view_name;
```

EXAMPLE

```
select * from student_details;  
desc student_details;
```

16.4 Updatable Views

Definition: Some views allow INSERT/UPDATE/DELETE directly through them, which then modifies the underlying base table.

SYNTAX

```
INSERT INTO view_name (col1, col2) VALUES (val1, val2);
```

RULES & NOTES

- A view is updatable when its rows can be mapped cleanly back to specific rows of an underlying base table. The most reliable case is a simple single-table view with no GROUP BY/HAVING/DISTINCT/aggregate functions — some multi-table/join views can also be updatable under MySQL's more detailed view-processing rules, but single-table is the dependable baseline to reason from.
- Whichever case applies, all NOT NULL columns without defaults in the underlying table must be included (or be nullable/defaulted) for INSERT to work.
- Inserting into a view actually inserts a new row into the real underlying table.

EXAMPLE

```
insert into student_details(stdid,name,class) values(111,"pavani",7);
```

16.5 DROP VIEW

Definition: Permanently removes a view definition (the underlying base table and its data are unaffected).

SYNTAX

```
DROP VIEW [IF EXISTS] view_name;
```

EXAMPLE

```
DROP VIEW IF EXISTS student_details;
```

16.6 AUTOCOMMIT (Background Concept)

Definition: By default, MySQL runs in autocommit mode: every single statement is automatically committed immediately after it runs.

SYNTAX

```
SHOW VARIABLES LIKE 'autocommit';  
SET autocommit = 0; -- turn off, to control commits manually
```

RULES & NOTES

- Use START TRANSACTION/BEGIN when you want to explicitly group several statements into one transaction — or alternatively, SET autocommit = 0 keeps every subsequent statement grouped into an ongoing transaction automatically, without needing to type BEGIN each time, until you COMMIT or ROLLBACK.

16.7 START TRANSACTION / BEGIN

Definition: Marks the start of a transaction block — a group of statements treated as a single unit of work.

SYNTAX

```
BEGIN;  
-- or  
START TRANSACTION;
```

RULES & NOTES

- Transactional data changes made after BEGIN remain part of the transaction until COMMIT or ROLLBACK. Statements that cause an implicit commit — many DDL statements (Module 3.2), for example — will end the current transaction early, even without an explicit COMMIT.

EXAMPLE

```
begin;  
update accounts set amount = amount-2000 where acc_no=101;  
update accounts set amount = amount+2000 where acc_no=105;
```

16.8 COMMIT

Definition: Permanently saves all changes made during the current transaction.

SYNTAX

```
COMMIT;
```

RULES & NOTES

- Once committed, changes cannot be undone with ROLLBACK.
- Calling ROLLBACK right after COMMIT has NO effect — the data is already permanently saved.

EXAMPLE

```
begin;  
update accounts set amount = amount-2000 where acc_no=101;  
update accounts set amount = amount+2000 where acc_no=105;  
commit;  
rollback; -- has no effect, already committed
```

16.9 ROLLBACK

Definition: Undoes all changes made in the current (not-yet-committed) transaction, restoring data to how it was before BEGIN.

SYNTAX

```
ROLLBACK;
```

RULES & NOTES

- Only works for uncommitted changes within the active transaction.

EXAMPLE

```
begin;  
update accounts set amount = amount-1000 where acc_no=103;  
rollback; -- undoes the update above
```

16.10 SAVEPOINT & ROLLBACK TO

Definition: A SAVEPOINT marks a named point within a transaction that you can roll back to, without undoing the entire transaction.

SYNTAX

```
SAVEPOINT savepoint_name;  
...  
ROLLBACK TO savepoint_name;
```

RULES & NOTES

- Only statements executed AFTER the savepoint are undone; statements before it remain intact (until a final COMMIT/full ROLLBACK).
- Useful for partial error recovery within a larger multi-step transaction.

EXAMPLE

```
begin;  
update accounts set amount = amount-1000 where acc_no=103;  
update accounts set amount = amount+1000 where acc_no=101;  
savepoint sp1;  
update accounts set amount = amount-1000000000 where acc_no=103; -- mistake  
rollback to sp1; -- undoes only the mistaken update, keeps the first two
```

Stored Procedures and Triggers

Packaging reusable SQL logic that runs on-demand (procedures) or automatically in response to data changes (triggers).

17.1 Introduction to Stored Procedures

Definition: A stored procedure is a saved block of SQL code (with logic, parameters, and control flow) that can be called by name — similar to a function/method in programming languages.

RULES & NOTES

- Encourages reuse (write the logic once, call it many times) and can encapsulate multi-step business logic server-side.

17.1b Advantages of Stored Procedures

Why you'd move logic into the database as a stored procedure instead of keeping it entirely in application code.

- Reusable SQL logic — write it once, call it from anywhere (Module 17.4), exactly like the `get_students_by_class` procedure shown earlier.
- **Reduces network traffic** — multiple operations run on the server in one round trip, instead of the application sending several separate queries back and forth.
- **Stores/centralizes business logic** — the rules live in one place (the database) rather than being duplicated across every application that touches the data.
- **Provides security/access control** — users can be granted permission to `CALL` a procedure without being granted direct access to the underlying tables it touches.
- **Reduces repeated parsing overhead** — MySQL maintains an internal representation and caching for stored programs, so repeated `CALLs` avoid re-parsing the SQL text from scratch every time (though it's not accurate to describe this as a fully "compiled executable" the way a C program is compiled).

17.2 DELIMITER

Definition: Temporarily changes the statement-terminator character (normally `;`) so that `;` can be used freely INSIDE a procedure body without ending the `CREATE PROCEDURE` statement early.

SYNTAX

```
DELIMITER //
-- procedure body using ; internally
DELIMITER ;
```

RULES & NOTES

- Must be reset back to `;` after the procedure definition, or every following statement will misbehave.

EXAMPLE

```
delimiter //
create procedure ...
begin
...
end //
delimiter ;
```

17.3 CREATE PROCEDURE Syntax — IN, OUT, INOUT Parameters

Definition: Defines a reusable named block of SQL logic that can accept parameters in three modes: **IN** (read-only input, the default), **OUT** (returns a value to the caller), and **INOUT** (both reads and returns a value).

SYNTAX

```

DELIMITER //
CREATE PROCEDURE procedure_name (
    IN param1 datatype,
    OUT param2 datatype,
    INOUT param3 datatype
)
BEGIN
    -- SQL statements
END //
DELIMITER ;

```

RULES & NOTES

- **IN** — value provided by the caller; changes inside the procedure don't affect the caller's variable.
- **OUT** — the caller passes a session variable (e.g. @count) which is populated after the CALL; its incoming value is ignored.
- **INOUT** — a two-way parameter: read AND updated in the same call (e.g. a counter/accumulator).

EXAMPLE — IN

```

create procedure get_students_by_class(in class_number int)
begin
select * from students where class=class_number;
end //

```

EXAMPLE — OUT

```

create procedure get_class_students_count(in class_number int,out total_count int)
begin
select count(*) into total_count from students where class=class_number;
end //

```

EXAMPLE — INOUT

```

CREATE PROCEDURE COUNTER(INOUT NUM INT)
BEGIN
SET NUM=NUM+1;
END //

```

17.4 Calling Procedures (CALL)

Definition: Executes (invokes) a stored procedure by name, passing any required arguments.

SYNTAX

```
CALL procedure_name(arg1, arg2, ...);
```

RULES & NOTES

- Argument count and order must match the procedure's parameter list.

EXAMPLE

```

CALL get_students_by_class(10);
SET @count=0;
CALL get_class_students_count(10,@count);
SELECT @count;

```

17.5 User/Session Variables & IF...ELSE Control Flow

Definition: Session variables (prefixed with @) persist for the current connection and are often used to capture OUT/INOUT results. IF...ELSE...END IF adds conditional branching inside a procedure body.

SYNTAX

```

SET @variable_name = value;
IF condition THEN
  -- statements
ELSE
  -- statements
END IF;

```

RULES & NOTES

- Session variables are not declared with a data type — MySQL infers it from the assigned value.
- IF blocks must always close with `END IF;` and can be extended with `ELSEIF`.

EXAMPLE

```

CREATE PROCEDURE EVEN_OR_ODD(IN NUM INT)
BEGIN
  IF NUM % 2 = 0 THEN
    SELECT "EVEN";
  ELSE
    SELECT "ODD";
  END IF;
END //
CALL EVEN_OR_ODD(10);

```

17.6 DROP PROCEDURE

Definition: Permanently deletes a stored procedure definition.

SYNTAX

```

DROP PROCEDURE [IF EXISTS] procedure_name;

```

RULES & NOTES

- Does not affect any tables or data the procedure used to operate on.

EXAMPLE

```

DROP PROCEDURE IF EXISTS get_students_by_class;

```

17.7 Introduction to Triggers — BEFORE/AFTER INSERT, UPDATE, DELETE

Definition: A trigger is a block of SQL code that MySQL automatically executes in response to a specific event (INSERT, UPDATE, or DELETE) happening on a table — no manual CALL needed. Timing (BEFORE/AFTER) controls whether it runs before or after the triggering event is applied.

SYNTAX

```

DELIMITER //
CREATE TRIGGER trigger_name
{BEFORE|AFTER} {INSERT|UPDATE|DELETE}
ON table_name
FOR EACH ROW
BEGIN
  -- statements
END //
DELIMITER ;

```

RULES & NOTES

- Every trigger must specify exactly one timing (BEFORE/AFTER) and one event (INSERT/UPDATE/DELETE) — a separate trigger is needed for each distinct event.
- **BEFORE** triggers are often used to validate/modify data before it's saved.
- **AFTER** triggers are often used for logging/auditing after the change is already applied (as shown below).

EXAMPLE

```
create trigger after_transaction_trigger
after update
on accounts
for each row
begin
insert into transactions(account,type,amount)
values(old.acc_no,if(old.amount>new.amount,"debit","credit"),abs(old.amount-new.amount));
end //
```

17.8 OLD and NEW Row References

Definition: Inside a trigger, OLD refers to the row's existing values, and NEW refers to the row's incoming values associated with the triggering operation. For an **AFTER** trigger these really are "before" and "after" a change that has already happened — but for a **BEFORE** trigger, NEW is really the value that's *about to be* written (and can even still be modified before it's saved), not something that has already occurred.

SYNTAX

```
OLD.column_name -- existing row value
NEW.column_name -- new/incoming row value for this operation
```

RULES & NOTES

- OLD is not available for INSERT triggers (no "before" row exists).
- NEW is not available for DELETE triggers (no "after" row exists).
- Both are available for UPDATE triggers, as in the example below.

EXAMPLE

```
insert into transactions(account,type,amount)
values(old.acc_no, if(old.amount>new.amount,"debit","credit"), abs(old.amount-new.amount));
```

17.9 FOR EACH ROW

Definition: Specifies that the trigger logic runs once for every individual row affected by the triggering statement (MySQL only supports row-level triggers, not statement-level).

SYNTAX

```
... FOR EACH ROW
BEGIN ... END;
```

RULES & NOTES

- If an UPDATE statement affects 5 rows, a FOR EACH ROW trigger runs 5 times, once per row.

EXAMPLE

```
-- this trigger fires once per row affected by the UPDATE
update accounts set amount = amount-200000 where acc_no = 103;
update accounts set amount = amount+200000 where acc_no=101;
select * from transactions;
```

17.10 Practical Use Cases (Auditing, Validation)

Definition: The two most common real-world reasons to use triggers: auditing (automatically logging every change for later review, as shown below) and validation (enforcing a business rule before allowing a change, using a BEFORE trigger).

EXAMPLE — AUDITING

```
-- every UPDATE on accounts automatically creates a matching transactions row
create trigger after_transaction_trigger after update on accounts for each row
begin
insert into transactions(account,type,amount)
values(old.acc_no, if(old.amount>new.amount,"debit","credit"), abs(old.amount-new.amount));
end //
```

EXAMPLE — VALIDATION (BEFORE TRIGGER PATTERN)

```
-- reject a withdrawal that would push amount negative
DELIMITER //
CREATE TRIGGER prevent_overdraft BEFORE UPDATE ON accounts
FOR EACH ROW
BEGIN
    IF NEW.amount < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient funds';
    END IF;
END //
DELIMITER ;
```

17.11 DROP TRIGGER

Definition: Permanently removes a trigger definition from a table.

SYNTAX

```
DROP TRIGGER [IF EXISTS] trigger_name;
```

RULES & NOTES

- The table and its data are unaffected — only the automatic behavior is removed.

EXAMPLE

```
DROP TRIGGER IF EXISTS after_transaction_trigger;
```

ER Modelling (Entities, Relationships, Diagrams)

The conceptual design layer that sits above everything else in this guide — every table built in classicmodels, school, and bank is really the physical implementation of an ER diagram like the ones in this module.

18.1 Entities & Attributes

Definition: An **entity** is a real-world object or concept that a database stores data about (e.g. a Customer, an Order, a Student). An **attribute** is a property/characteristic of that entity (e.g. a Customer's name, city, or phone number).

RULES & NOTES

- An entity becomes a TABLE; an attribute becomes a COLUMN.
- A **key attribute** (underlined in ER diagrams) uniquely identifies each instance of the entity — it becomes the table's PRIMARY KEY.
- An **entity instance** is one specific occurrence of the entity (e.g. the customer "Atelier graphique") — it becomes one ROW.

EXAMPLE

```
Entity: CUSTOMER
Attributes: customerNumber (key), customerName, city, country, creditLimit
-- maps directly to a real table:
CUSTOMERS(customerNumber PK, customerName, city, country, creditLimit, ...)
```

18.2 Relationships & Cardinality

Definition: A **relationship** describes how two entities are associated (e.g. a Customer "places" an Order). **Cardinality** describes how many instances of one entity can relate to how many instances of another — expressed as One-to-One (1:1), One-to-Many (1:M), or Many-to-Many (M:N).

RULES & NOTES

- **1:1** — one row in A relates to at most one row in B (e.g. one Employee has one Office assignment record).
- **1:M** — one row in A relates to many rows in B, but each row in B relates to only one row in A (e.g. one Customer places many Orders; each Order belongs to exactly one Customer). This is implemented with a FOREIGN KEY on the "many" side.
- **M:N** — many rows in A relate to many rows in B (e.g. many Orders contain many Products). This is implemented with a linking/junction table holding a composite key (e.g. ORDERDETAILS).

EXAMPLE

```
CUSTOMER (1) ----places----> (M) ORDER          -- 1:M, FK on ORDERS.customerNumber
ORDER      (M) ----contains--> (M) PRODUCT        -- M:N, resolved via ORDERDETAILS
EMPLOYEE   (1) ----reportsTo--> (M) EMPLOYEE      -- self-relationship: one manager (1) can have many direct reports (M)
```

18.3 Keys in ER Modelling

Definition: How PRIMARY KEY and FOREIGN KEY concepts (Module 4.6-4.7) show up visually in an ER diagram.

RULES & NOTES

- Primary key attributes are underlined inside the entity box.
- A 1:M relationship line always places the resulting FOREIGN KEY on the "many" side's table.
- An M:N relationship is normally resolved using a junction/linking entity whose primary key is commonly the composite of both original keys (Module 5.5b) — though a separate surrogate key can also be used if the design calls for it.

18.3b Attribute Types: Simple, Composite, Multi-Valued, Derived

Definition: Not every attribute is a plain single value — ER modelling distinguishes four kinds, each drawn differently on a diagram.

RULES & NOTES

- **Simple attribute** — an ordinary, single, indivisible value (e.g. salary, date of birth). Drawn as a plain oval.
- **Composite attribute** — an attribute that's actually made up of smaller sub-parts (e.g. Name = first + middle + last).

Drawn as an oval connected to smaller sub-ovals for each part.

- **Multi-valued attribute** — an attribute that can hold more than one value for a single entity instance at once (e.g. an employee's mobile — someone might have two phone numbers). Drawn as a **double-bordered oval**.
- **Derived attribute** — an attribute whose value is calculated from another attribute rather than stored directly (e.g. age is derived from date of birth). Drawn as a **dashed oval**.

These four categories map directly onto real column-design decisions: a multi-valued attribute like mobile can't just be one column — it needs its own related table (a normalization call straight out of Module 12's 1NF rule against repeating groups). A derived attribute like age often isn't stored as a column at all — it's calculated on the fly, exactly like the `FLOOR(DATEDIFF(NOW(), dob)/365)` pattern from Module 11.7.

18.3c Weak Entities

Definition: A **weak entity** is an entity that cannot be uniquely identified by its own attributes alone — it depends on a related "owner"/parent entity's key to be identified. Drawn as a **double-bordered rectangle**.

RULES & NOTES

- A weak entity's primary key is typically a composite of the owner entity's key plus a partial key of its own — the same pattern as a composite primary key (Module 5.5b).

18.4 Notation

Definition: The diagram below uses a simple, widely-taught notation: rectangles for entities, a bold key attribute listed first (underlined), and connecting lines labeled with "1" or "M" at each end to show cardinality.

RULES & NOTES

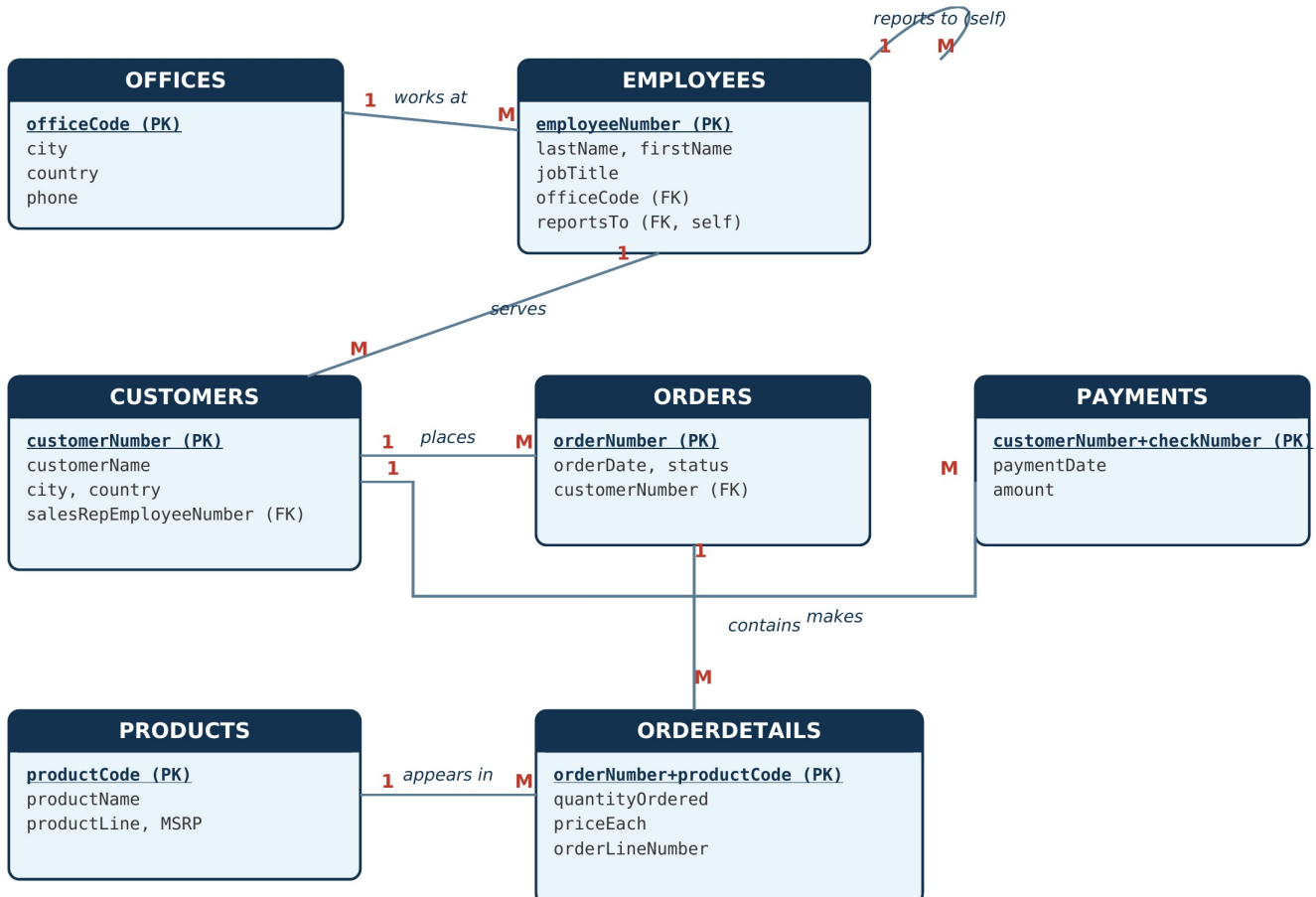
- This is a simplified hybrid of Chen notation (entity boxes) and Crow's Foot cardinality labels (1/M at line ends) — the style most commonly taught alongside MySQL.

18.5 Full ER Diagram of the Classicmodels Database

Two worked examples: the full classicmodels schema, and an Employees/Department diagram redrawn in full notation.

DIAGRAM 1 — FULL CLASSICMODELS SCHEMA

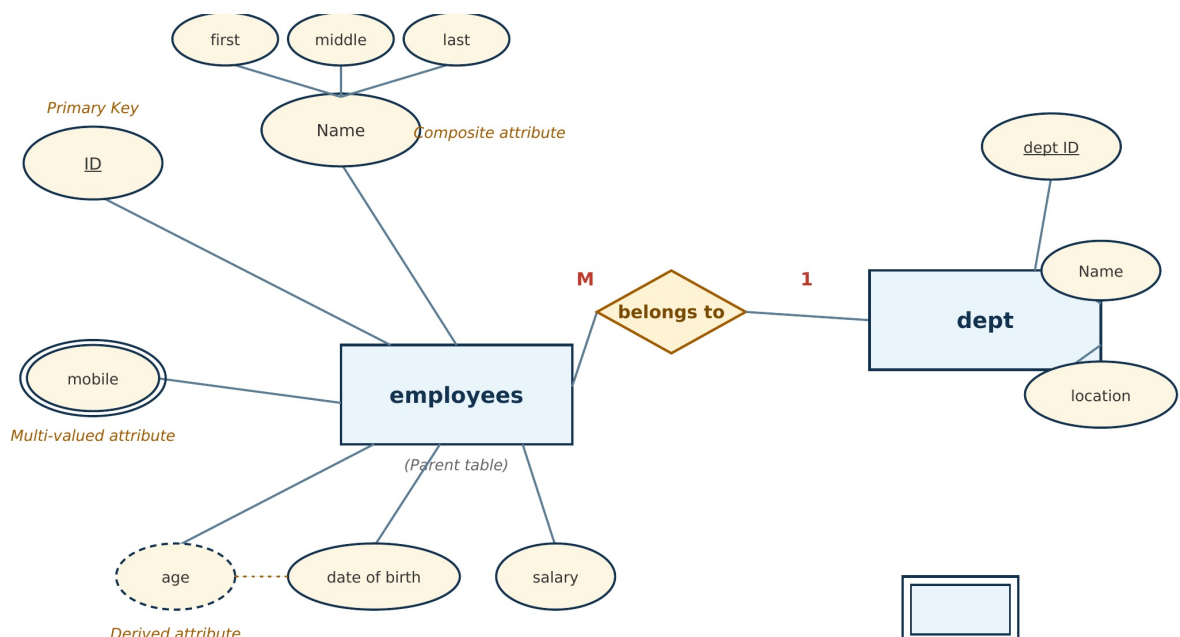
Built entirely from the classicmodels tables used across `Join.sql`, `Set.sql`, and `Views.sql` — seven entities and seven relationship connections, matching a real classicmodels schema.



Reading the diagram: an **OFFICE** has many **EMPLOYEES** (1:M); an **EMPLOYEE** can manage other employees, so **EMPLOYEE** has a self-relationship for reportsTo (the SELF JOIN in Module 14.6); an **EMPLOYEE** serves many **CUSTOMERS** as their sales rep (1:M); a **CUSTOMER** places many **ORDERS** (1:M) and can also make many **PAYMENTS** (1:M); an **ORDER** contains many **PRODUCTS**, and a **PRODUCT** can appear on many **ORDERS** — that Many- to-Many is resolved by the **ORDERDETAILS** junction entity, whose primary key is the composite (orderNumber, productCode) from Module 5.5b.

DIAGRAM 2 — EMPLOYEES & DEPARTMENT WORKED EXAMPLE

This worked example, redrawn in full Chen notation, demonstrates the attribute types from Module 18.3b (composite, multi-valued, derived) and the weak-entity notation from 18.3c that the classicmodels diagram above doesn't happen to use.



Reading this diagram: **employees** is the parent table, with **ID** as its underlined primary key. **Name** is a *composite attribute* built from first, middle, and last. **mobile** is a *multi-valued attribute* (double-bordered oval) — one employee can have more than one number. **age** is a *derived attribute* (dashed oval), calculated from **date of birth** rather than stored directly. **salary** and **date of birth** are ordinary simple attributes. The **belongs to** relationship is **M:1** — many employees belong to one department — and **dept** has its own attributes: dept ID, Name, location. The small double-bordered rectangle is the general notation for a *weak entity* (Module 18.3c) — shown here as a standalone legend, not as part of this specific employees/dept relationship.